AI-Powered Cardiovascular Patient Monitoring System V 1.0

Ehsan Hayat

Adeel Ahmed

shehroz khan

SUPERVISOR NAME:

DR. Amir Qayyum

Fall-2025

Department of Computer Science

Capital University of Science & Technology, Islamabad

Table of Contents

List of Tables

List of Figures

Chapter 1 :Introduction

1.1 Project Introduction

This project presents a web-based Cardiovascular Patient Monitoring System designed to manage patient information, visualize physiological data, and support remote health monitoring through a centralized dashboard. The system is implemented using PHP and MySQL and communicates with external devices using REST-based APIs. Raspberry Pi devices are supported as monitoring units for data transmission.

The current implementation focuses on user management, patient records, device management, and dashboard-based visualization of vital data. Advanced features such as medical-grade sensor integration (ADS1292R) and AI-based disease prediction are planned for future phases.

1.2 Existing Examples / Solutions

Existing cardiovascular monitoring solutions such as Holter monitors, Apple Watch ECG, and mobile ECG devices provide basic heart monitoring capabilities. However, these systems are often expensive, closed-source, or lack integration with hospital information systems. The proposed system aims to provide a flexible, low-cost, and extensible academic prototype.

Table 1 1.2.1 Comparison of Existing Cardiovascular Monitoring Systems

Sr. No.	System	AI Integration	Real-Time Monitoring	Cost Efficiency	Hospital Integration
1	Holter Monitor	✗	✓	✗	✓
2	Apple Watch ECG	✓	✓	✗	✗
3	Mobile	✓	✓	✗	✗
4	Proposed System	✓	✓	✓	✓

1.3 Business Scope

The system can be used in hospitals, clinics, telemedicine platforms, and home-based monitoring. It supports digital health record management and can be extended for remote patient monitoring and future AI-assisted decision support.

1.4 Useful Tools and Technologies

Proposed Technology Stack

The current implementation uses PHP for backend development, MySQL for database management, and HTML, CSS, and JavaScript for frontend interfaces. REST APIs are used for data communication. Raspberry Pi devices act as monitoring gateways. AI frameworks and advanced sensors are considered future enhancements.

1.5 Project Work Breakdown

The project is divided into requirement analysis, system design, software development, testing, and documentation. Hardware upgrades and AI integration are planned in later stages. A sample work breakdown structure is illustrated below.

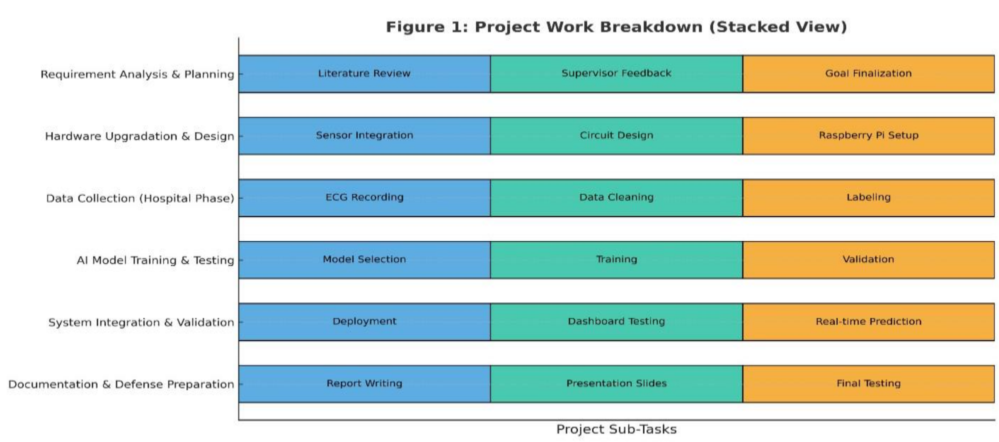


Figure 1 1.5.1 project work breakdown

1.6 Project Timeline

The following table represents the estimated project timeline, showing the distribution of tasks across weeks.

Table 2 1.2.1 Project timeline

Phase	Duration	Key Activities
Requirement Analysis & Planning	Oct 2025	Supervisor feedback, literature review, planning hospital visits
Hardware Upgradation & Design	Nov – Dec 2025	Purchase & integrate new sensors, test signal quality
Data Collection (Hospital Phase)	Jan – Feb 2026	Collect ECG readings from 60–70 patients at RIC
Data Labeling & Preprocessing	Mar 2026	Label and clean collected ECG data
AI Model Training & Testing	Apr – May 2026	Train models, evaluate performance, tune parameters
System Integration & Validation	Jun 2026	Deploy model into system, test real-time predictions
Final Documentation & Defense Preparation	Jul 2026	Report writing, final testing, and FYP defense prep

Chapter 2:Requirement Specification and Analysis

Requirements analysis is the process of determining and documenting the expectations for the proposed system's improvements. Since the project is upgrading to the medical-grade **ADS1292R** and implementing a multi-class AI, these requirements are rigorous, focusing on clean data acquisition and accurate disease classifications.

2.1 Functional Requirements

Functional Requirements are the specific actions the system must be able to perform.

Table 3 2.1.1 Functioonal Requirements

S No	Functional Requirement	Type	Status
1	Secure User Registration & Authentication	Core	Pending
2	Patient Profile & Medical History Management	Core	Pending
3	Role-Based Access Control (RBAC)	Core	Pending
4	Real-Time Dashboard Visualization (ECG, HR, SpO_2, Respiration)	Core	Pending
5	Device Registration & Management (Raspberry Pi)	Core	Pending
6	Upgraded Sensor Data Acquisition (ADS1292R – 24-bit ECG/Respiration)	Core	Pending
7	Real-Time Data Pre-processing using ADS1292R	Core	Pending
8	Multi-Class AI Prediction Service	Intermediate	Pending
9	Critical Alert Generation (Edge)	Core	Pending
10	AI Prediction History & Auditing	Intermediate	Pending
11	Data Export (EHR Compatibility)	Intermediate	**Planned**

2.2 Non-Functional Requirements

Non-Functional Requirements are about the **quality** of the system—how reliable, fast, and secure it is.

Table 4 2.2.1 Non Functional requirements

S No	Non-Functional Requirements	Category
1	The system shall enforce secure authentication and role-based access control to protect patient data.	Security
2	Secure communication between users and the web system shall be supported using standard web security practices (HTTPS depending on deployment).	Security
	The system dashboard shall respond within an acceptable time under normal operating conditions.	Performance
3	The system shall be reliable for continuous monitoring during normal usage.	Reliability
4	The system architecture shall support future scalability for AI integration and additional devices.	Scalability
5	The system shall be usable on standard web browsers with a simple and intuitive interface.	Usability
6	Advanced AI accuracy metrics and real-time latency constraints are defined as target goals for future phases.	AI Performance (Planned)
7	The system shall enforce secure authentication and role-based access control to protect patient data.	Security

2.3. Selected Functional Requirements

Following is the list of the requirements selected for the current iteration.

Table 5 2.3.1 Selelcted Functional Requirements

S. No.	Functional Requirement	Type
1	Upgraded Sensor Data Acquisition using ADS1292R (24-bit ECG, Respiration)	Core
2	Real-Time Signal Pre-processing on Raspberry Pi	Core
3	Multi-Class AI Prediction Service	Core

4	Critical Alert Generation at Edge Device	Core
5	Enhanced Real-Time Dashboard Visualization (ECG, HR, SpO_2, Respiration)	Intermediate
6	Patient Medical History with AI Prediction Logs	Intermediate

NOTE :

“Only User Management, Patient Records, Device Management, And Dashboard Visualization Are Implemented In The Current Phase. The Following Requirements Represent The Planned Implementation Roadmap.”

2.4. System Use Case Modeling

The system includes Admin, Doctor, and Patient actors. Use cases focus on authentication, patient management, device management, and data visualization.

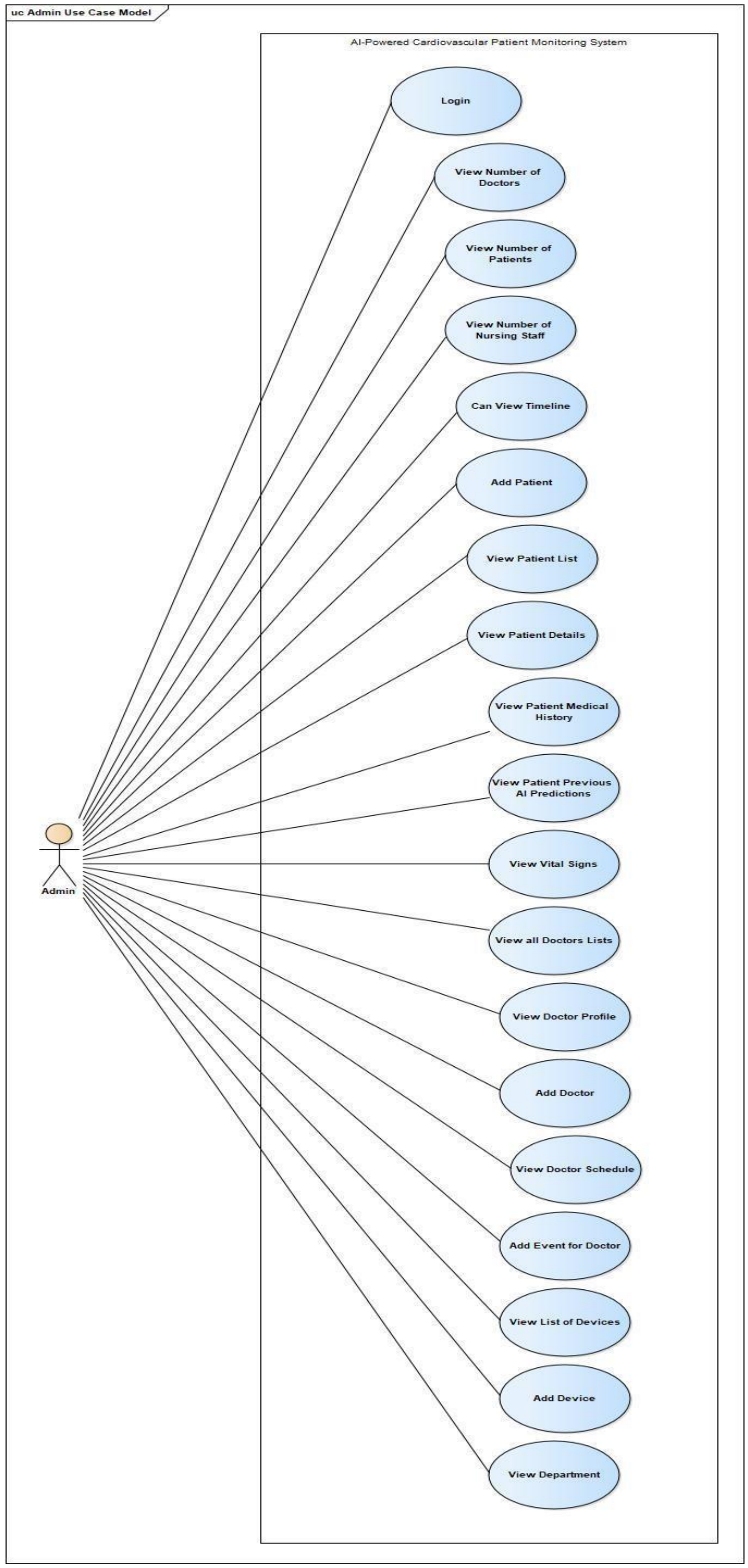


Figure 2 2.4.1 Admin Usecase model

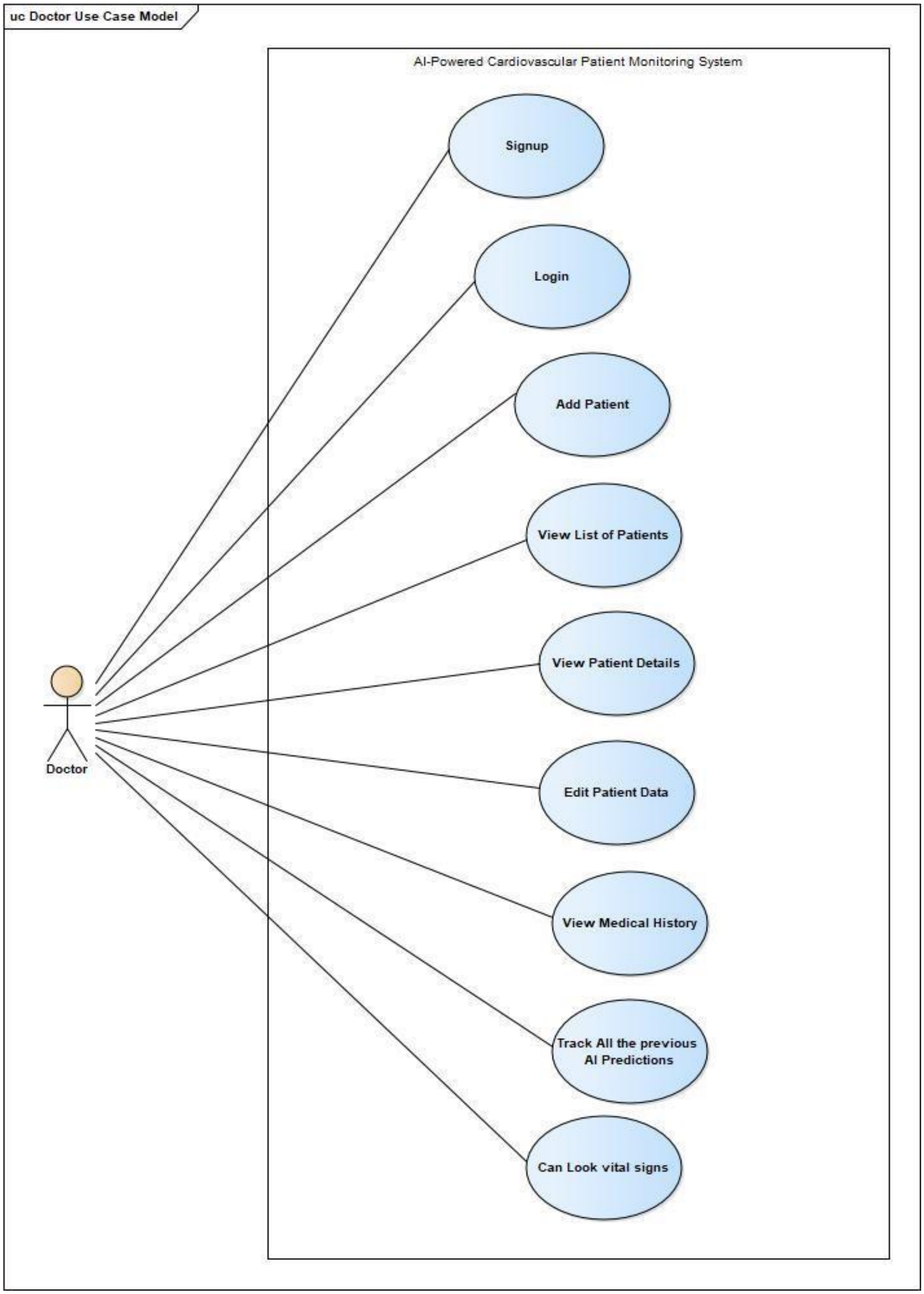


Figure 3 2.4.2 Doctors Usecase model

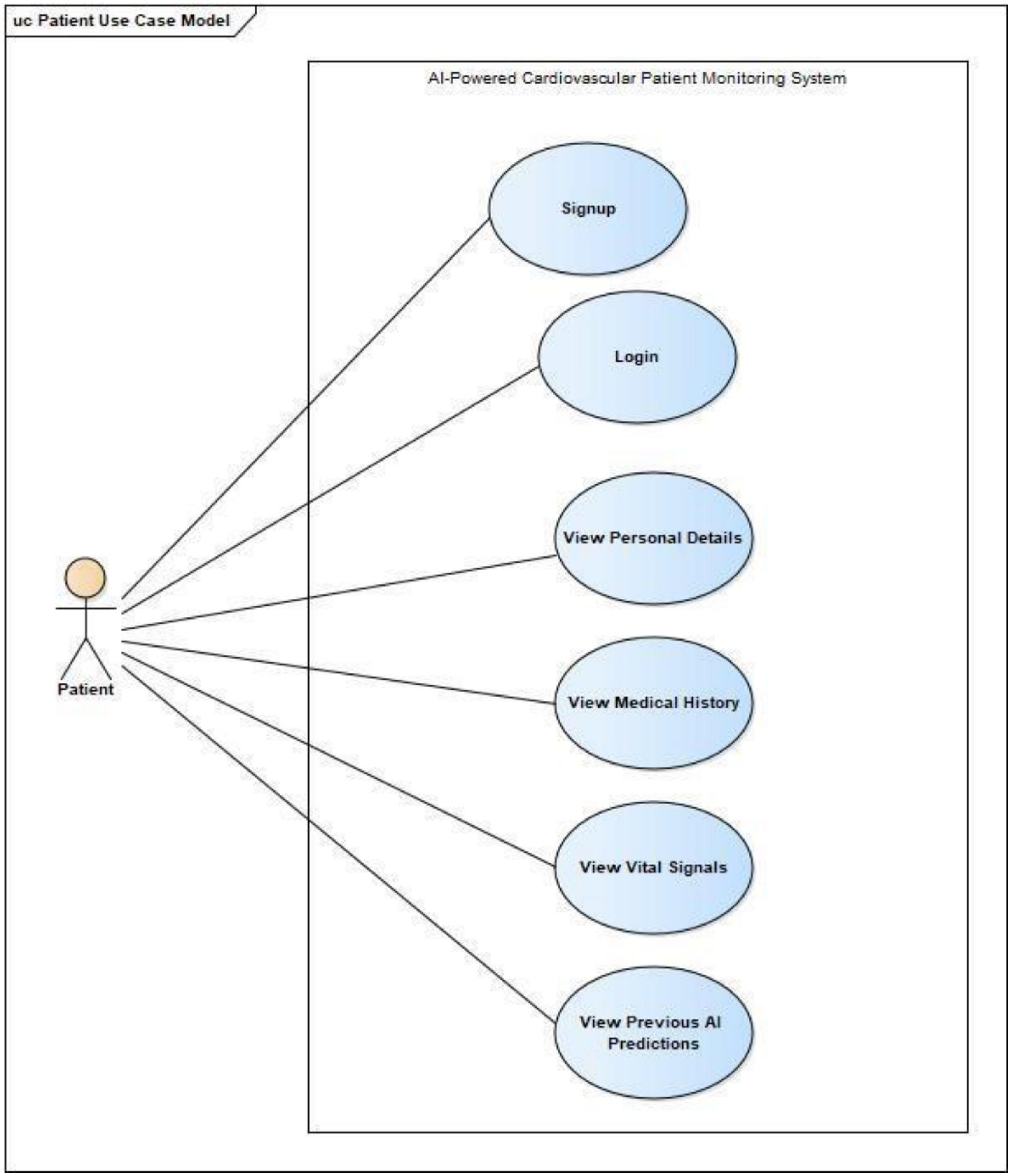


Figure 4 2.4.3 Patient usecase model

Table 6 2.4.1 USECASE

Table 7 2.4.1

Use Case ID:	**Uc1**		
Use Case Name:I	User Signup.		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Patient		
Description:	The Patient creates a new account to access personal health data and alerts.		
Trigger:	Patient clicks sign up.		
Preconditions:	Patient has no account.		
Postconditions:	Account created.		
Normal Flow:	**Patient**	**System**	
	Initiates action.	Displays signup form.	
	Enters name, mobile number, and password.	Validates input format.	
	Submits data.	Encrypts password, stores data, and responds with confirmation.	
Alternative Flows:	The Patient cancels the operation. / The System auto-populates address based on location.		
Exceptions:	Invalid input format (e.g., weak password). / Database connection error.		

Table 8 2.4.2

Use Case ID:	Uc2		
Use Case Name:l	Login		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Patient		
Description:	The Patient securely accesses the system using valid credentials.		
Trigger:	The Patient clicks the 'Login' button.		
Preconditions:	The Patient has an existing, active account.		
Postconditions:	The Patient is authenticated and redirected to the personal dashboard.		
Normal Flow:	**Patient**	**System**	
	Initiates action.	Displays login form.	
	Enters username and password.	Validates credentials against stored data.	
	Submits data.	Generates access token and redirects to the dashboard.	
Alternative Flows:	The Patient uses 'Forgot Password' link.		
Exceptions:	Invalid username or password. / Server timeout during authentication.		

Table 9 2.4.3

Use Case ID:	**Uc3**		
Use Case Name:I	View Vital Signals		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Patient		
Description:	The Patient views the real-time stream of physiological data (ECG, Respiration, SpO2) collected by their personal device.		
Trigger:	The Patient selects the 'View Vital Signals' option on the dashboard.		
Preconditions:	The Patient is logged in; the Monitoring Device is online and streaming data.		
Postconditions:	The system displays a live graphical view of the Patient's vital signs.		
Normal Flow:	**Patient**	**System**	
	Requests to view vital signs.	Authenticates the request and fetches the live stream from the Edge Gateway.	
	Views live ECG waveform and numerical readings (HR, SpO2, Respiration Rate).	Displays data and continuously updates the graphs (FR-6.0).	
Alternative Flows:	The Patient filters the view to show only the last 24 hours of readings.		
Exceptions:	The Monitoring Device goes offline (System displays 'Offline'). / No historical data is found.		

Table 10 2.4.4

Use Case ID:	**Uc4**		
Use Case Name:I	View previous AI predictions		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Patient		
Description:	The Patient reviews the historical records of the system's automated multi-class disease predictions.		
Trigger:	The Patient selects the 'View Previous AI Predictions' option.		
Preconditions:	The Patient is logged in; the AI prediction log exists for the patient.		
Postconditions:	The system displays a chronological log of all past AI diagnoses, scores, and timestamps.		
Normal Flow:	**Patient**	**System**	
	Requests the prediction history.	Retrieves the AIPredictionLog from the database (FR-8.0).	
	Views the list of prediction classes (e.g., 'Normal', 'Tachycardia') and confidence scores.	Displays the log, highlighting any 'Critical Alerts' (FR-5.0).	
Alternative Flows:	The Patient filters the log by date range or prediction class.		
Exceptions:	No AI predictions are recorded. / Database query error.		

Table 11 2.4.5

Use Case ID:	**Uc5**		
Use Case Name:l	View Medical History		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Patient		
Description:	The Patient views non-real-time medical summaries and associated doctor information.		
Trigger:	The Patient selects the 'View Medical History' tab.		
Preconditions:	The Patient is logged in.		
Postconditions:	The system displays patient demographic information and associated Doctor details.		
Normal Flow:	**Patient**	**System**	
	Requests medical history.	Fetches static medical records and assigned Doctor details (FR-7.0).	
	Reviews summary and Doctor's contact information.	Displays the fetched information securely.	
Alternative Flows:	The Patient prints or exports the record summary (FR-10.0).		
Exceptions:	Database retrieval fails. / No medical history is recorded.		

Table 12 2.4.6

Use Case ID:	**Uc6**		
Use Case Name:I	Add Patient		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Doctor		
Description:	The Doctor registers a new patient into the monitoring system and assigns them to an available device.		
Trigger:	The Doctor clicks the 'Add Patient' button.		
Preconditions:	The Doctor is logged in; the Doctor has permission to add patients.		
Postconditions:	A new patient record is created and linked to the Doctor's profile.		
Normal Flow:	**Doctor**	**System**	
	Requests to add patient.	Displays patient creation form.	
	Enters patient details (name, date of birth, contact, etc.).	Validates data.	
	Submits details.	Creates patient entry and confirms registration.	
Alternative Flows:	The Doctor attempts to assign the patient to a device already in use. $\rightarrow$ System warns the Doctor and requests a different device ID.		
Exceptions:	Invalid or missing mandatory fields. / Database write error.		

Use Case ID:	**Uc7**		
Use Case Name:I	Track All Previous AI Predictions		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Doctor		
Description:	The Doctor reviews the comprehensive historical AI diagnosis log for any assigned patient, including prediction classes and confidence scores.		
Trigger:	The Doctor selects a patient and clicks 'Track All previous AI Predictions'		
Preconditions:	The Doctor is logged in and authorized for the patient; prediction data exists.		
Postconditions:	A sortable, filterable table of the patient's full prediction history is displayed (FR-8.0).		
Normal Flow:	**Doctor**	**System**	
	Selects the patient.	Verifies authorization (FR9.0).	
	Requests full prediction log.	Retrieves all AIPredictionLog records for the Patient ID.	
	Filters the view to show only high-confidence abnormal readings.	Displays filtered data in a sortable log format.	
Alternative Flows:	The Doctor exports the prediction history log (FR-10.0).		
Exceptions:	The Doctor is not authorized to view the patient. / Database access fails.		

Table 13 2.4.7

Use Case ID:	Uc8		
Use Case Name:I	Can Look Vital Signs (Real-Time Monitoring)		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Doctor		
Description:	The Doctor views the real-time multi-parameter data streams (ECG, Respiration, SpO2) of an active patient for immediate clinical assessment.		
Trigger:	The Doctor selects a patient from the 'View List of Patients' dashboard.		
Preconditions:	The Doctor is logged in; the Patient's Monitoring Device is actively streaming data (FR-2.0).		
Postconditions:	A live dashboard showing all vital sign graphs and the latest AI prediction is displayed (FR-6.0).		
Normal Flow:	**Doctor**	**System**	
	Selects patient for real-time viewing.	Fetches current status and opens the monitoring dashboard.	
	Monitors live ECG waveform, Respiration Rate, and SpO2 trends.	Displays dynamic graphs and instantaneous AI prediction updates (NFR-3.0).	
	Observes a critical alert flash.	Logs alert acknowledgement when the Doctor clicks on it (FR-5.0).	
Alternative Flows:	The Doctor pauses the live data stream to inspect a segment of the ECG waveform more closely.		
Exceptions:	The live data stream connection fails. / The Doctor is unauthorized to view the patient.		

Table 14 2.4.8

Table 15 2.4.9

Use Case ID:	Uc9		
Use Case Name:I	Add Doctor / Add Event for Doctor		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Admin		
Description:	The Admin manages the healthcare workforce by adding new Doctor accounts or scheduling new events/appointments for them.		
Trigger:	The Admin selects 'Add Doctor' or 'Add Event for Doctor' from the management menu.		
Preconditions:	The Admin is logged in and authorized (FR-9.0).		
Postconditions:	A new Doctor account is created, or an event/schedule entry is added to the Doctor's record.		
Normal Flow:	**Admin**	**System**	
	Requests to add Doctor or add event.	Displays the corresponding input form.	
	Enters required credentials (for new doctor) or schedule details (for event).	Validates inputs and updates the User and Doctor tables.	
	Submits data.	Confirms successful creation/update.	
Alternative Flows:	The Admin modifies an existing Doctor's details or schedule.		
Exceptions:	A Doctor with that ID already exists. / Invalid date/time format for the event.		

Table 16 2.4.10

Table 2.10: view Doctor Profile Usecase Description

Use Case ID:	**Uc10**			
Use Case Name:I	View Doctor Profile / View all Doctors Lists			
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat	
Date Created:	06-11-25	Last Revision Date:	06-11-25	
Actors:	Admin			
Description:	The Admin views a list of all registered doctors and can drill down to view individual Doctor profiles and schedules.			
Trigger:	The Admin selects 'View all Doctors Lists'.			
Preconditions:	The Admin is logged in.			
Postconditions:	The list of Doctors or a specific Doctor's profile is displayed.			
Normal Flow:	**Admin**	**System**		
	Requests the Doctor list.	Retrieves all Doctor records.		
	Views the list and selects a specific Doctor.	Displays the selected Doctor's profile, including schedule and managed patients.		
Alternative Flows:	The Admin views records for other staff ('View Number of Nursing Staff').			
Exceptions:	Database retrieval fails. / No Doctors are currently registered.			

Table 17 2.4.11

Table 2.11: Add Device Usecase Description

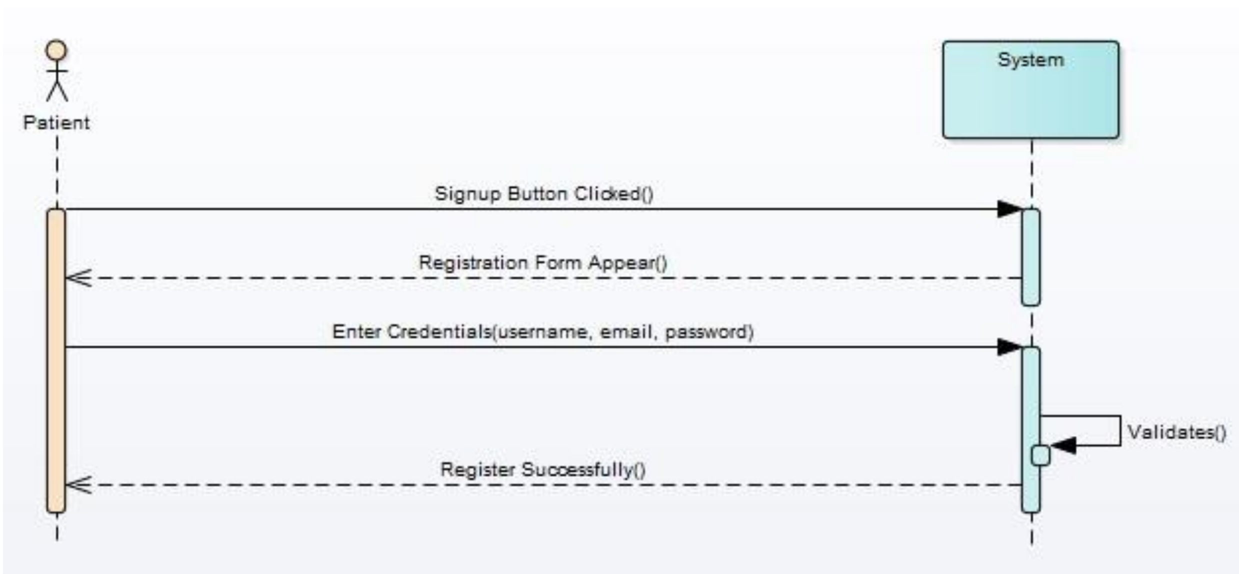
Use Case ID:	**Uc11**		
Use Case Name:I	Add Device / View List of Devices		
Created By:	Ehsan Hayat	Last Updated By:	Ehsan Hayat
Date Created:	06-11-25	Last Revision Date:	06-11-25
Actors:	Admin		
Description:	The Admin registers new Monitoring Devices (Raspberry Pi units) into the system inventory and manages their status.		
Trigger:	The Admin selects 'Add Device' or 'View List of Devices'.		
Preconditions:	The Admin is logged in.		
Postconditions:	A new device is registered, or the inventory list is displayed.		
Normal Flow:	**Admin**	**System**	
	Requests to add device.	Displays device registration form.	
	Enters device serial number and attributes.	Validates and stores the new device record.	
	Views the updated device list.	Retrieves and displays all device records, including online/offline status.	
Alternative Flows:	The Admin modifies the status or assigned patient of an existing device.		
Exceptions:	Device serial number already exists. / Database write error.		

2.5. System Sequence diagram

Sequence diagrams illustrate interactions for login, patient registration, device assignment, and dashboard access.

Patient Account Registration

Figure 5 2.5.1 Patient usecaseSSDs

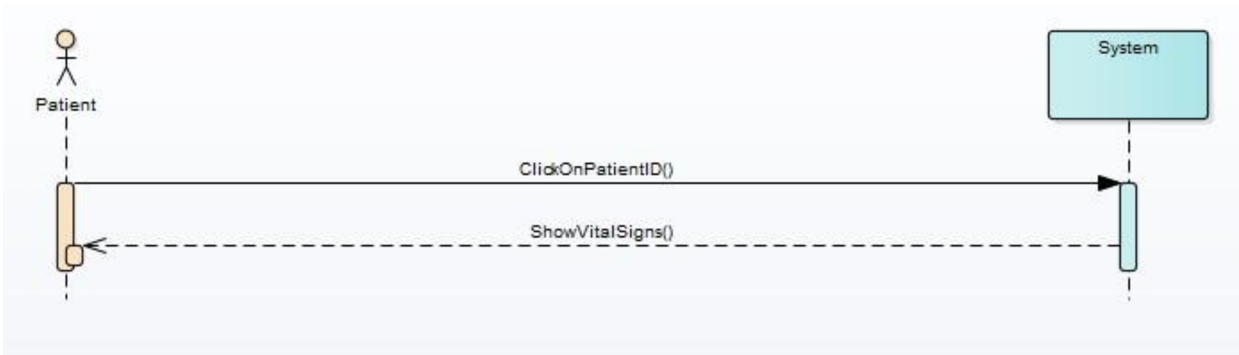


Patient System Login

Figure 6 2.5.2 patient login SSDS

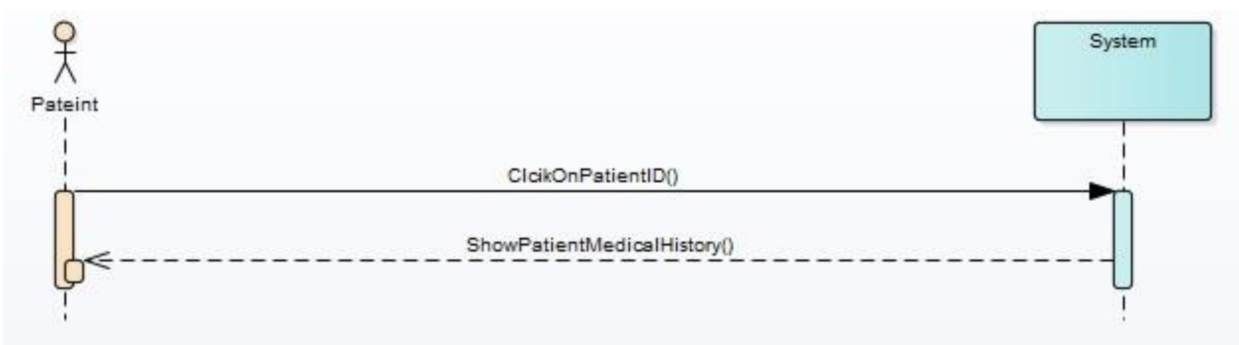
Patient View Vital Signs

Figure 7 2.5.3 Patient vital sign view



Patient View Medical History

Figure 8 2.5.4 Patient medical history view

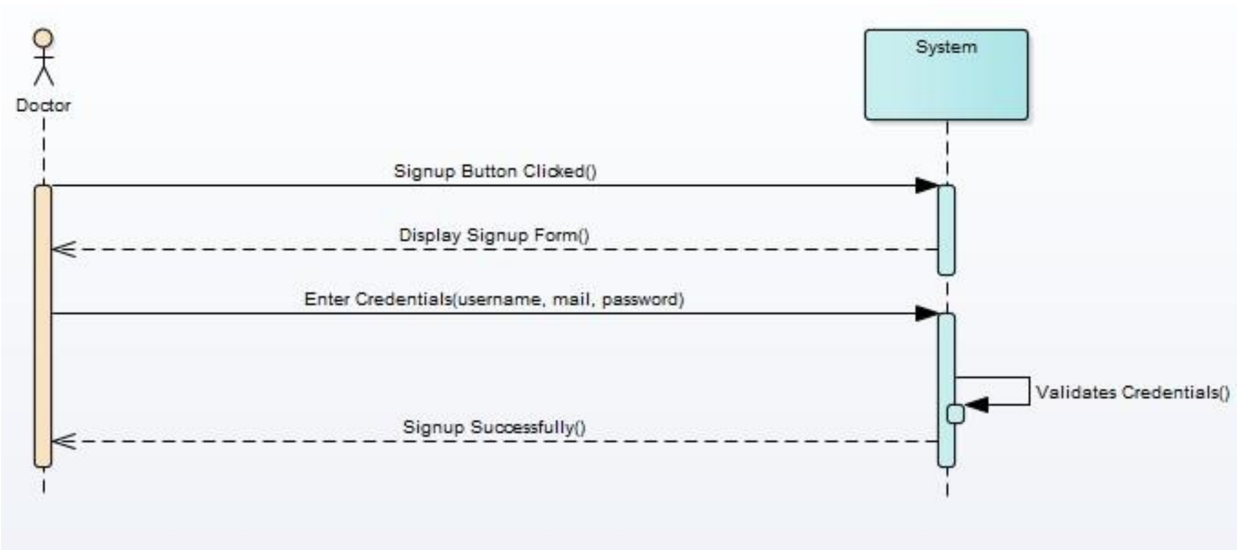


Patient View Previous AI Predictions

Figure 9 2.5.5 patient preious view history

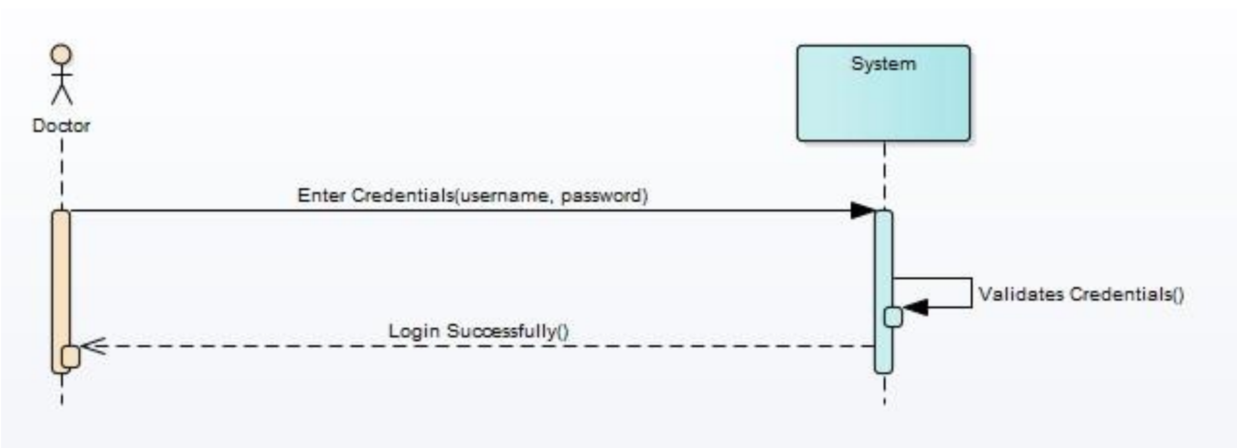
Doctor Account Registration

Figure 10 2.5.6 Doctors Registration SSD



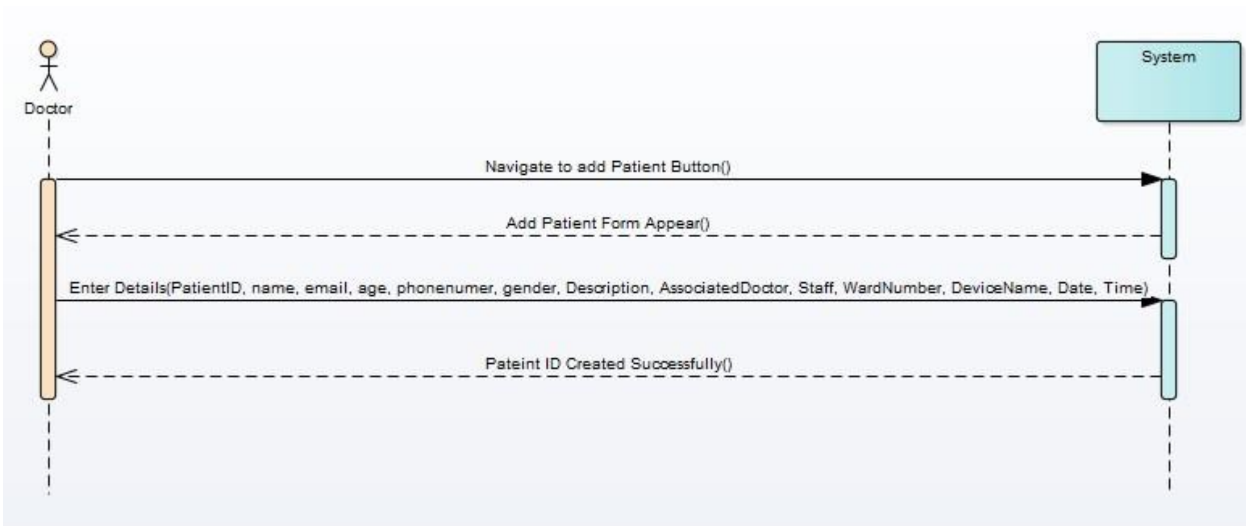
Doctor System Login

Figure 11 2.5.7 Doctors login SSDs



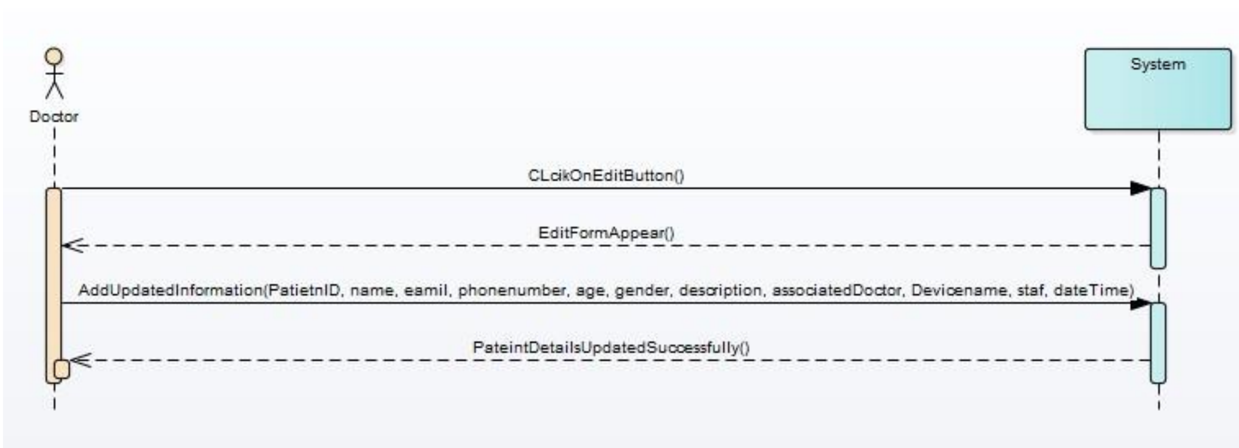
Doctor Add Patient

Figure 12 2.5.8 Doctor add patients SSD



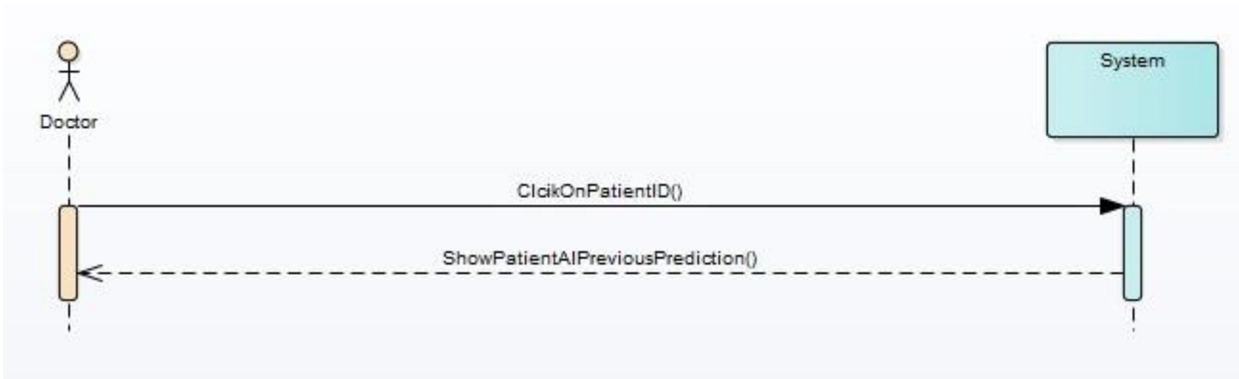
Doctor Edit Patient Data

Figure 13 2.5.9 Doctor edits patient SSDs



Doctor Track All AI Predictions

Figure 14 2.5.10 Doctor show previous patients history



Doctor View List of Patients

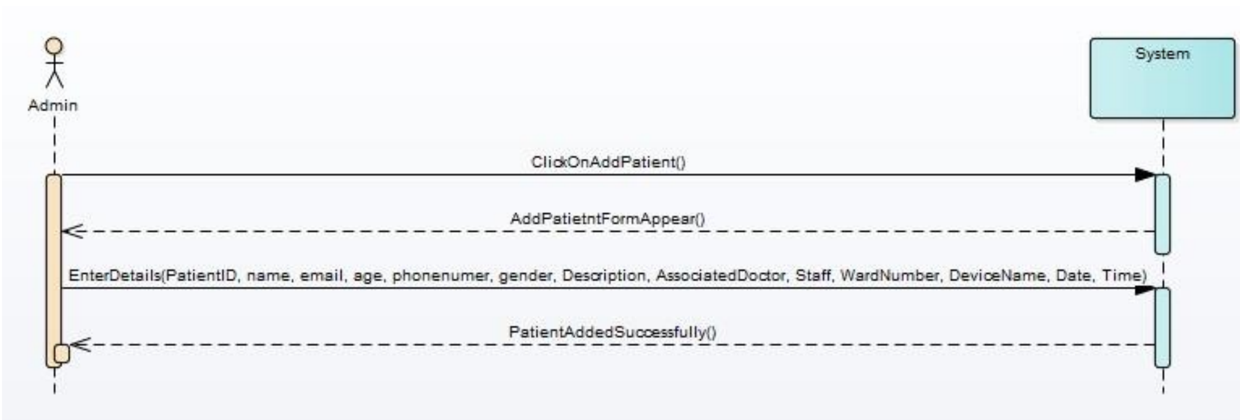
Figure 15 2.5.11 Doctor view list of patients

Admin System Login

Figure 16 2.5.12 Admin login

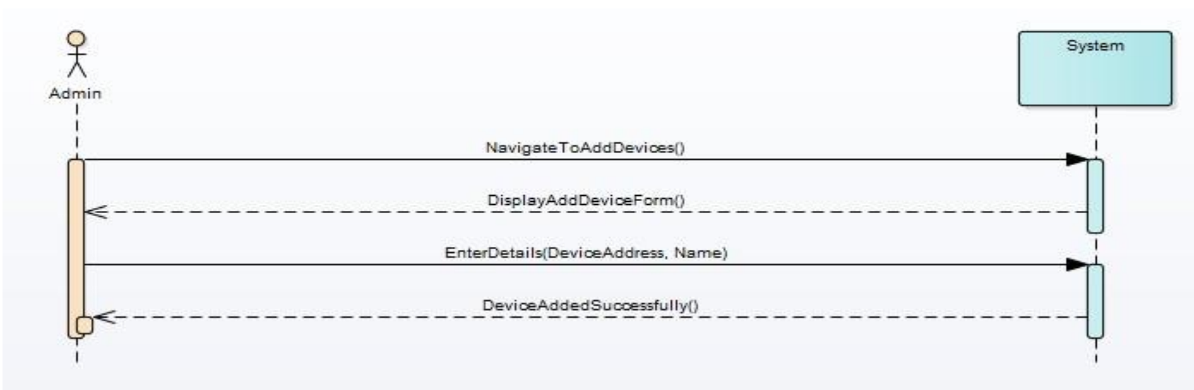
Admin Add Patient

Figure 17 2.5.13 Admin add patients



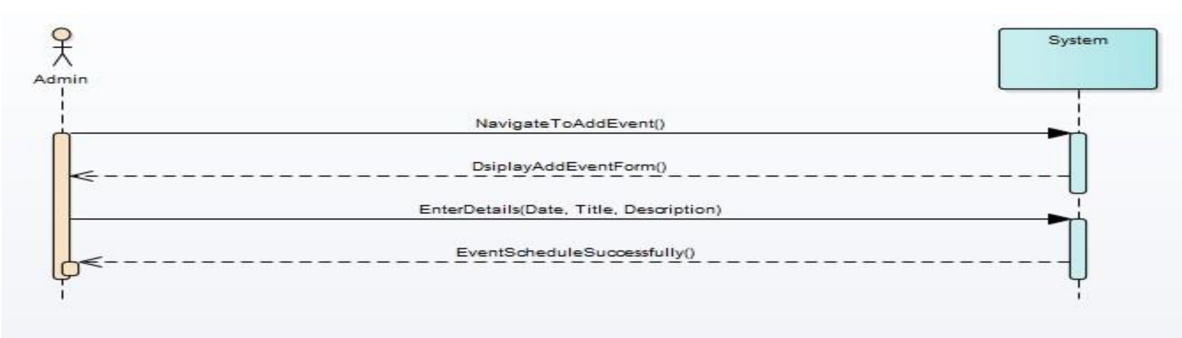
Admin Add Device

Figure 18 2.5.14 Admin add devices



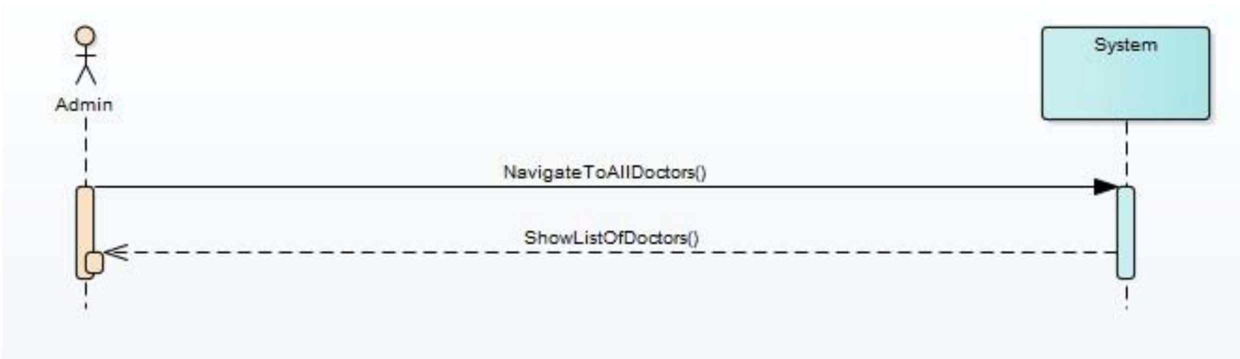
Admin Add Doctor Event/Schedule

Figure 19 2.5.15 admin add doctor event schedule



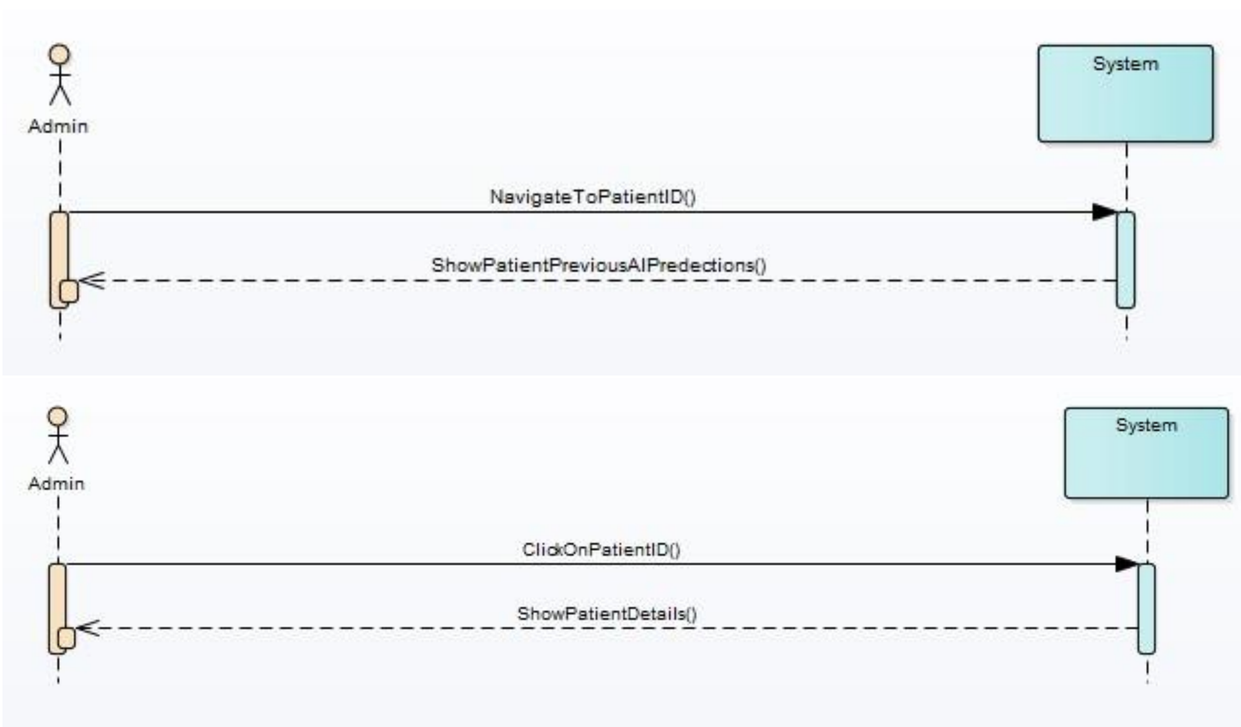
Admin View All Doctors List

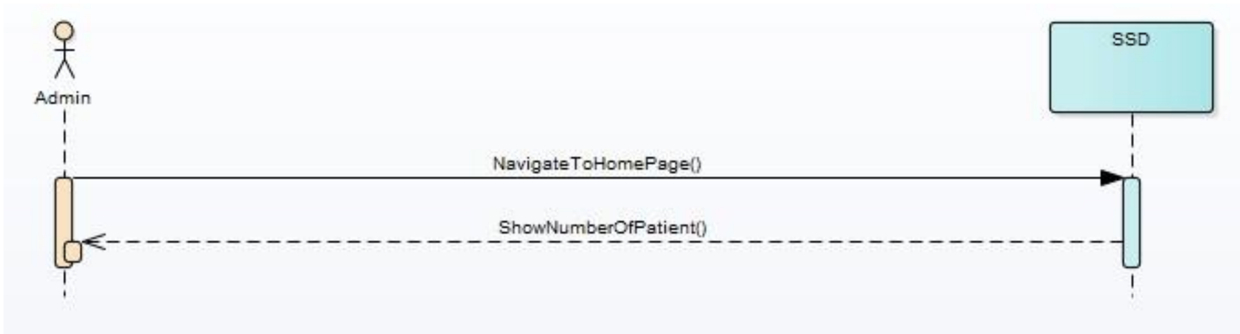
Figure 20 2.5.16 Admin can view doctor list

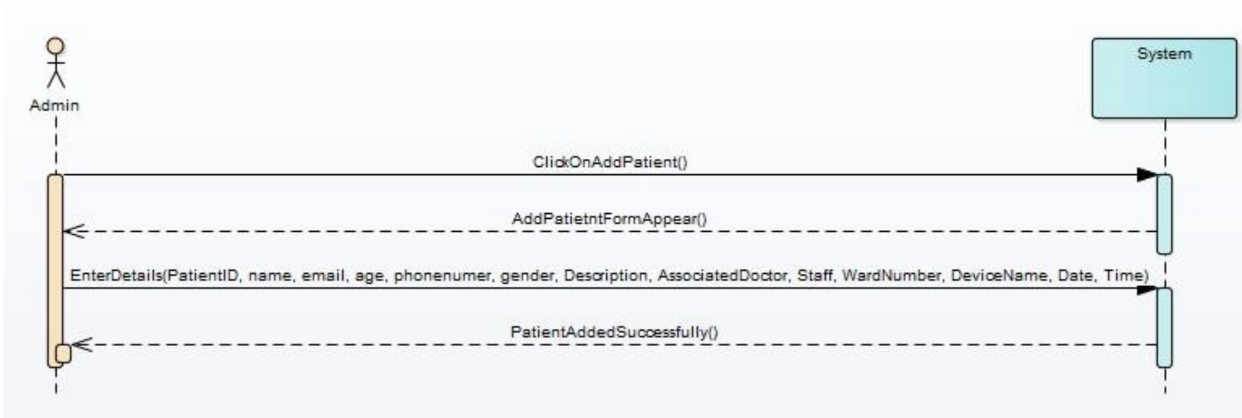


Admin View Patient List & Count

Figure 21 2.5.17 Admin patient list and count

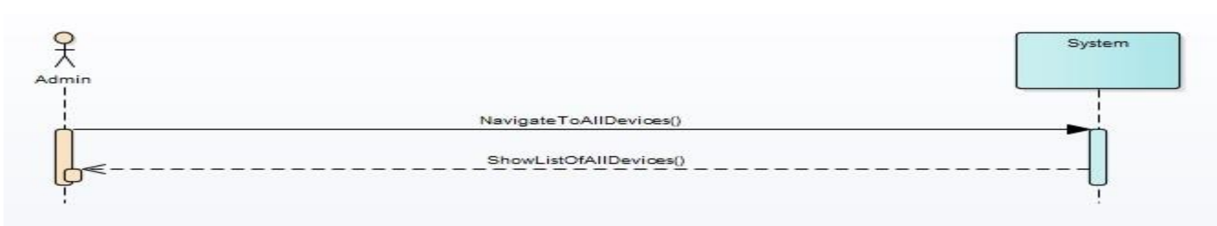


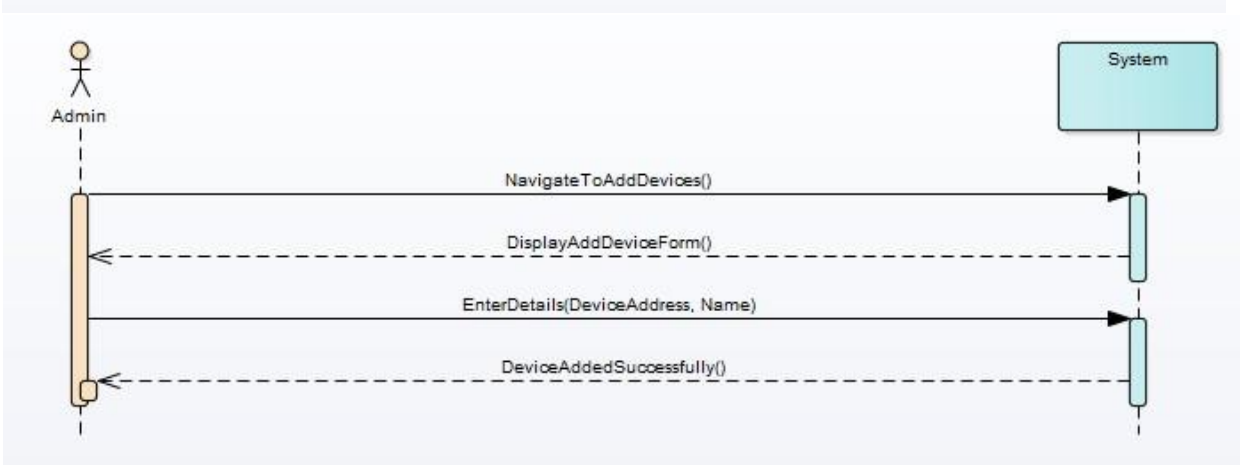




Admin View All Devices List

Figure 22 2.5.18 admin view all devices list

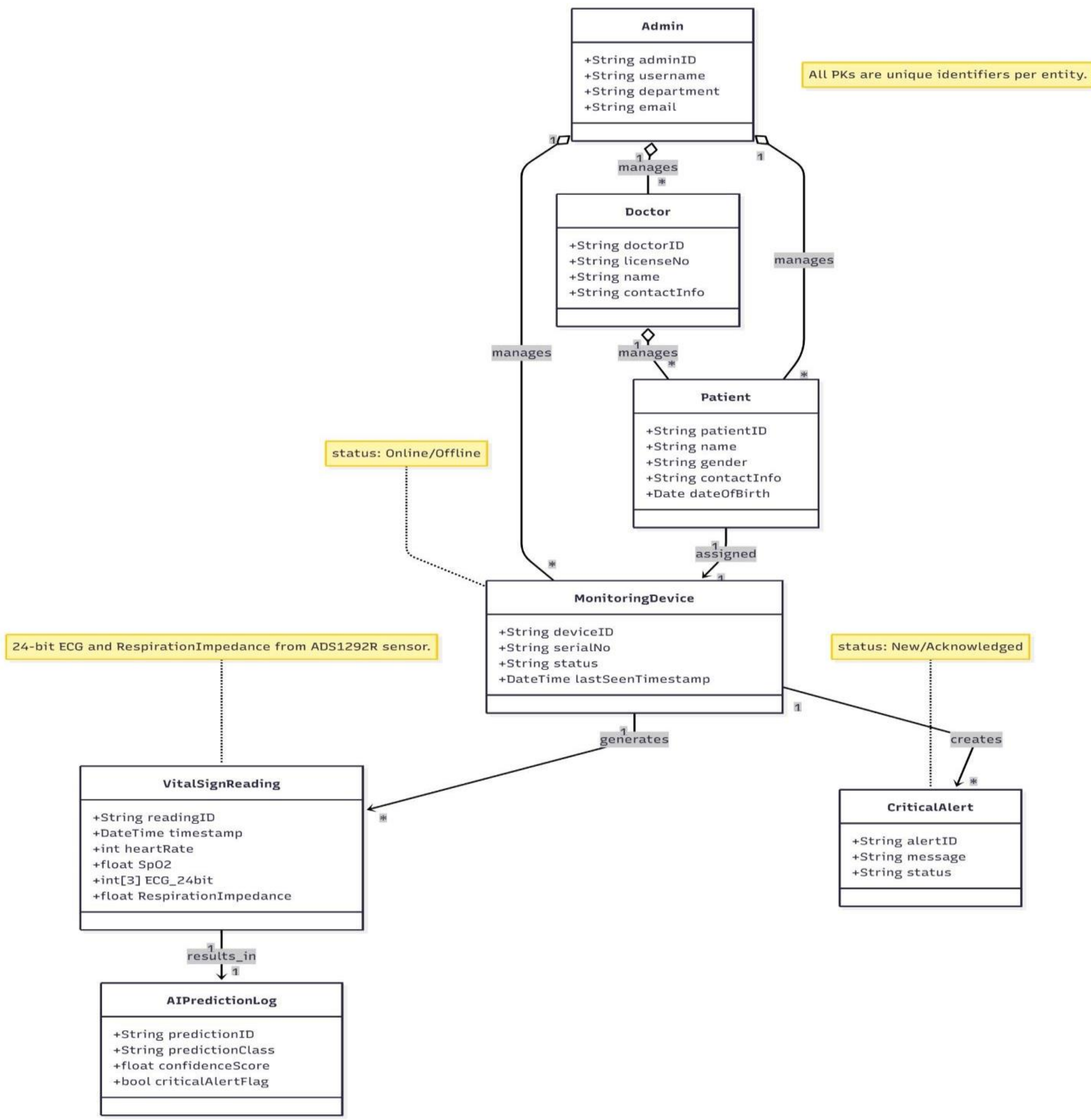




2.6. Domain Model

The domain model defines core entities such as User, Patient, Doctor, Device, and Vital Data. AI-related entities are reserved for future integration.

Figure 23 2.6.1 Domain Model



Chapter 3: System Design

3.1. Layer Definition

The system follows a three-tier architecture consisting of Presentation Layer, Business Logic Layer, and Database Layer.

Table 18 3.1 LAyers Defination

Layers	Description
Presentation Layer	This layer will be used for the interaction with the user through a graphical user interface (The React.js Dashboard) to display real-time data (**FR-6.0**).
Business Logic Layer	This layer contains the core application logic (Django Backend). All the constraints, user authentication (**FR-9.0**), and data routing functions reside under this layer.
Database Layer	This layer contains the database servers (PostgreSQL/MongoDB) where all permanent patient information and historical logs (**FR-8.0**) are stored and retrieved securely.

3.1.1. Presentation Layer

This tier is the top level that displays information and interacts with the user via a responsive graphical interface.

3.1.2. Business Logic Layer

Also called the middle tier, this layer controls application functionality by performing detailed processing, enforcing security rules, and managing communication between the database and the presentation layer.

3.1.3. Database Layer

This tier includes database servers where information is stored and retrieved. Data in this tier is kept independent of application servers, ensuring robust data integrity and security.

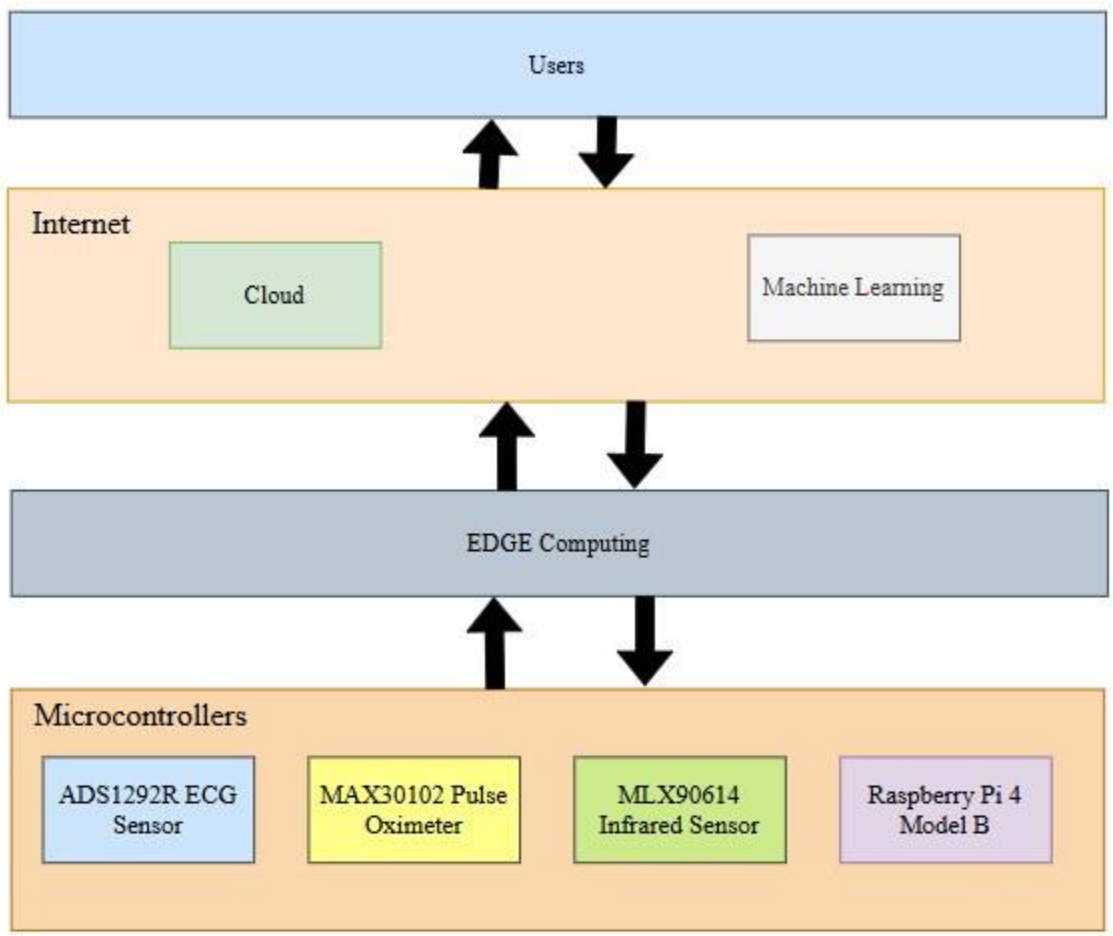
3.2. Software Architecture

The architecture is a web-based system integrated with IoT data sources via APIs. PHP handles backend logic, while MySQL stores persistent data. AI processing is planned as a future enhancement.

- **Architecture:** Hybrid Edge-Cloud Architecture.

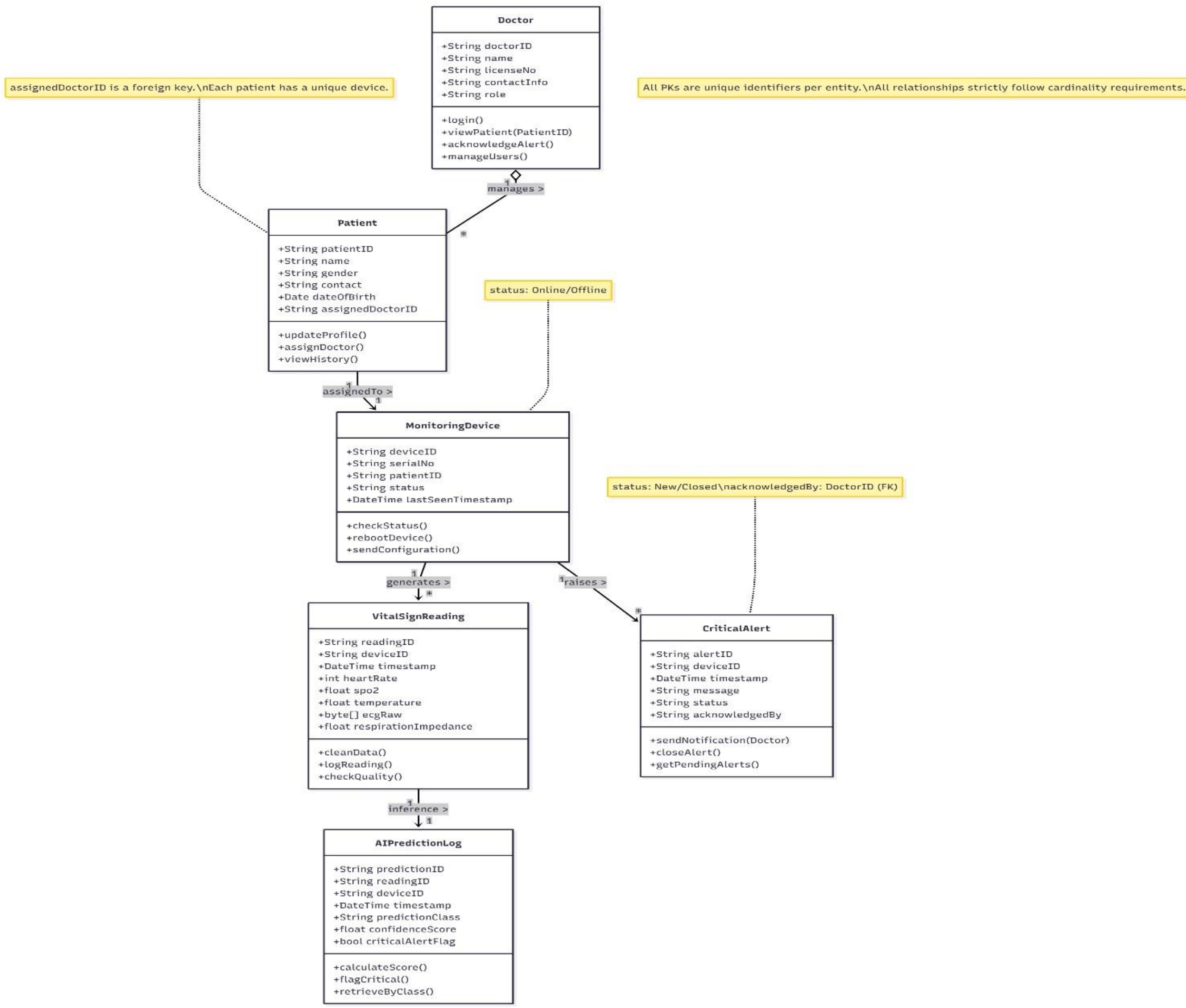
- **Sensing Network:** The **ADS1292R (24-bit ECG/Respiration)** and **MAX30102** (SpO_{2}) acquire the data.
- **Edge Processing Node (Raspberry Pi 4):** Performs local pre-processing and AI Inference (FR-3.0, FR-4.0).
- **Cloud API Gateway:** Secure entry point for all data and user requests.

Figure 24 3.2.1 Software Architecture



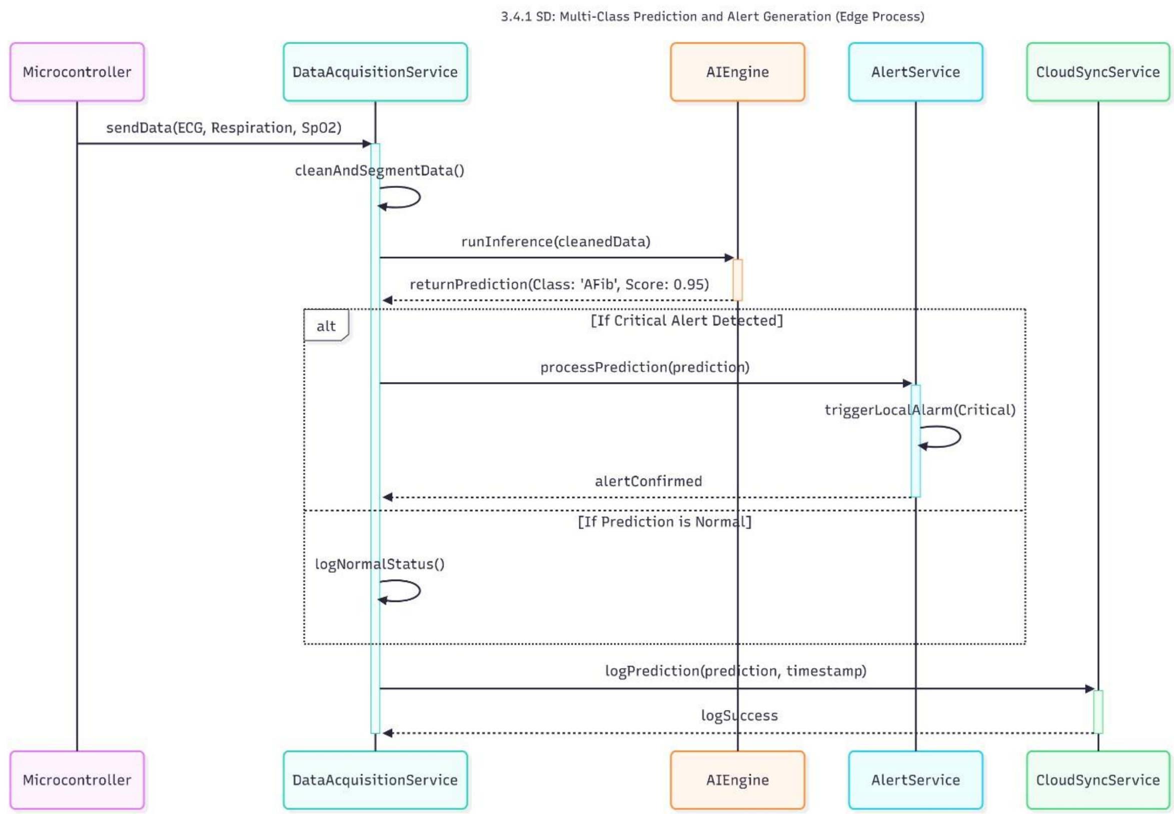
3.3. Class Diagram

Figure 25 3.3.1 Class Diagram



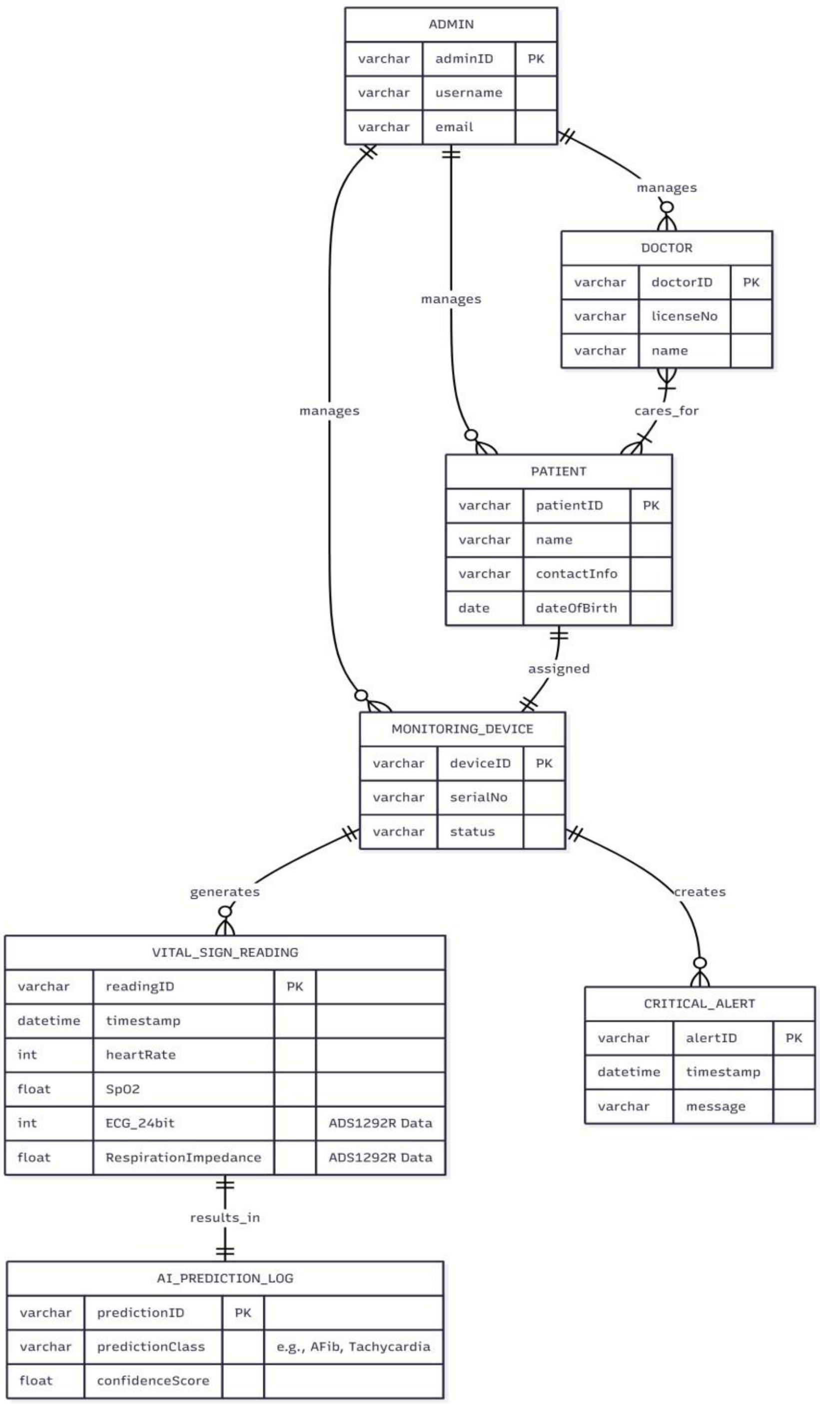
3.4. Sequence Diagram

Figure 26 3.4.1 Sequence Diagram



3.5. Entity Relationship Diagram

Figure 27 3.5.1 ERD

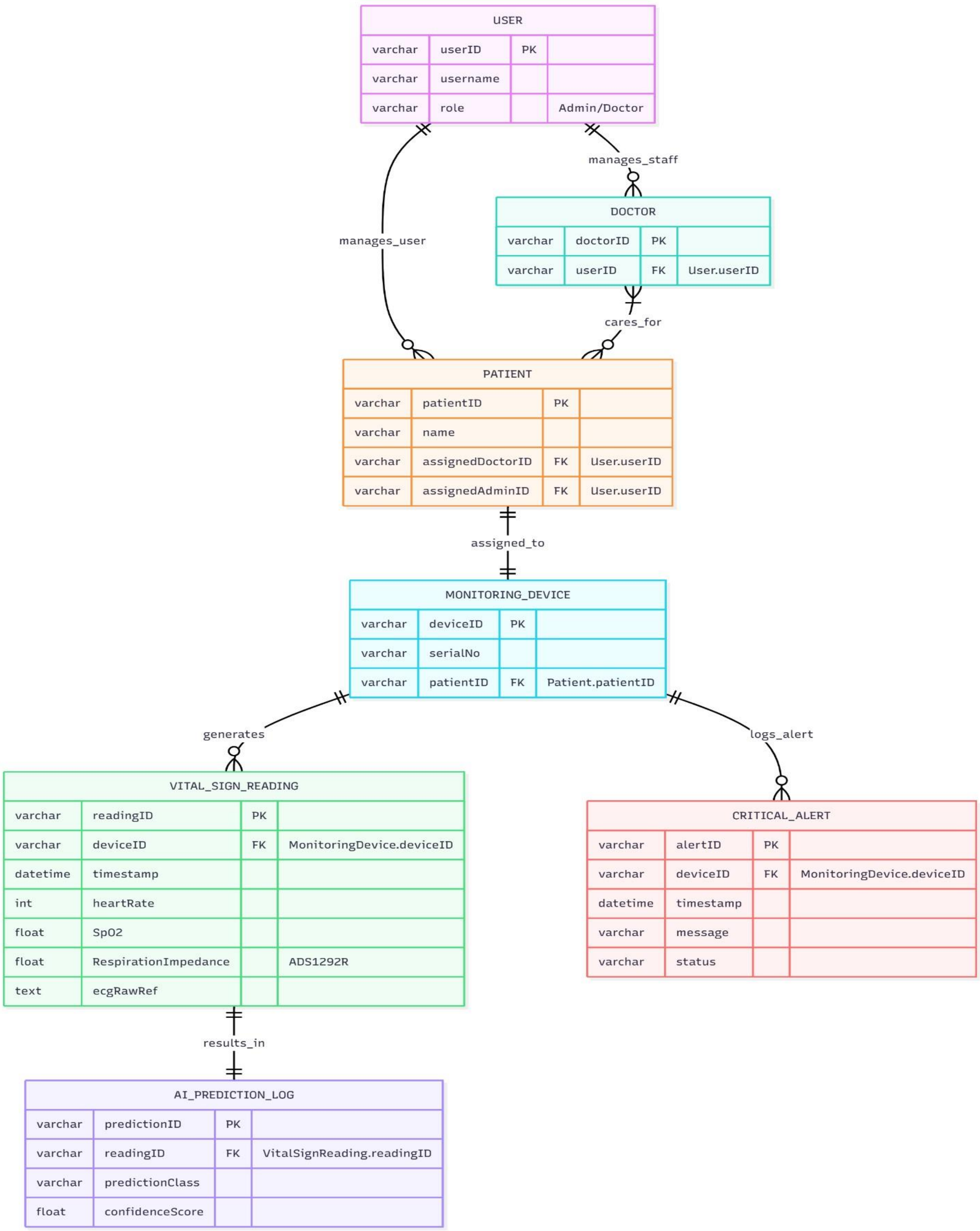


3.6. Database Schema

The database schema uses MySQL to store users, patients, doctors, devices, and vital data. Support for AI prediction logs is planned.

- **Tables:** Represented as boxes, each table corresponds to an entity in the system and includes a set of attributes. For this project, key tables include `PATIENT`, `VITAL_SIGN_READING`, and `AI_PREDICTION_LOG`.
- **Primary Key (PK):** A unique identifier for each record in a table. It ensures that no two rows have the same value for the primary key field (e.g., `patientID`).
- **Foreign Key (FK):** A field in one table that refers to the primary key in another table. It establishes a relationship between two tables, such as linking a `VITAL_SIGN_READING` record back to its `MONITORING_DEVICE`.
- **Relationships:** Lines connecting tables represent logical relationships such as One-to-One, One-to-Many, and Many-to-Many. These relationships define how data in one table relates to data in another.
- **Data Types and Constraints:** Columns include data types (e.g., `VARCHAR`, `INT`, `DATETIME`) and constraints (e.g., `NOT NULL` or explicit references to specialized types like **24-bit ECG** data) to ensure data validity and integrity across the system.

Figure 28 3.6.1 Database Schema

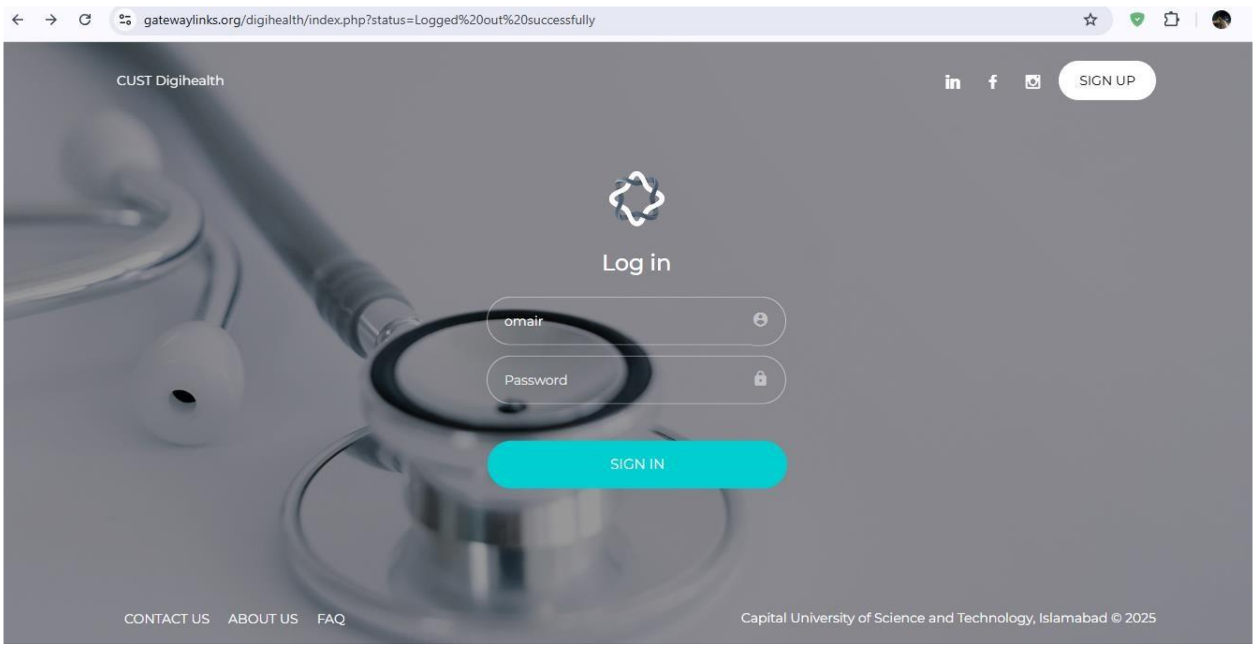


3.7. User Interface Design

The UI design outlines the key screens, prioritizing clarity and the immediate visibility of the new data streams.

Login page

Figure 29 3.7.1 login page



Sign UP page :

Figure 30 3.7.2 signup page

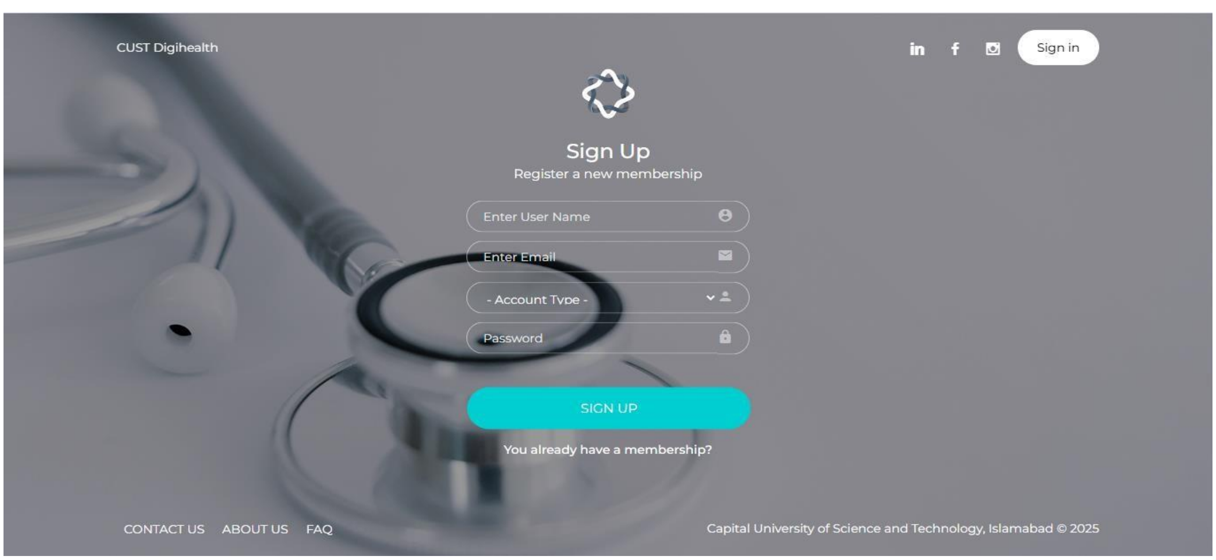
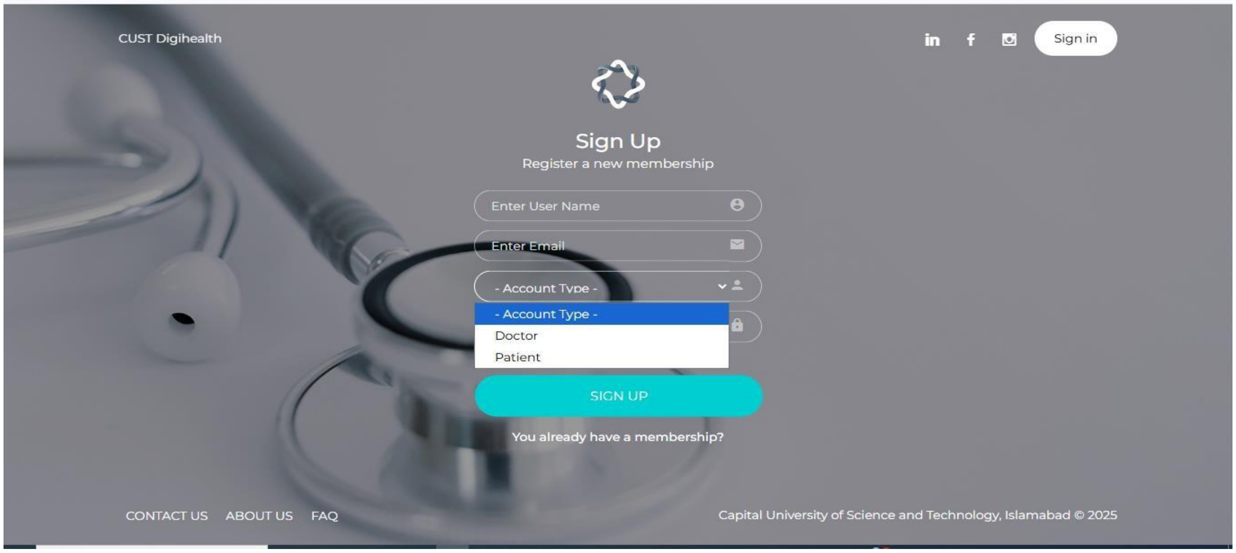


Figure 31 3.7.3 signup page



Admin's Dashboard:

Figure 32 3.7.4 Admins Dashboard

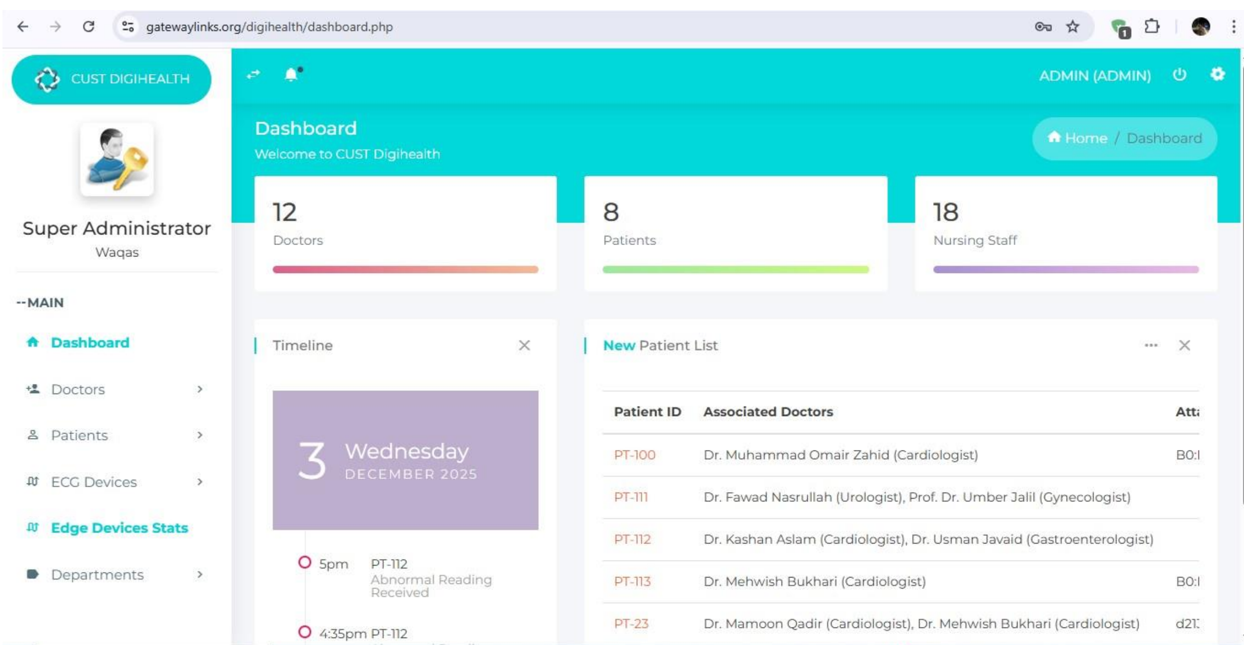
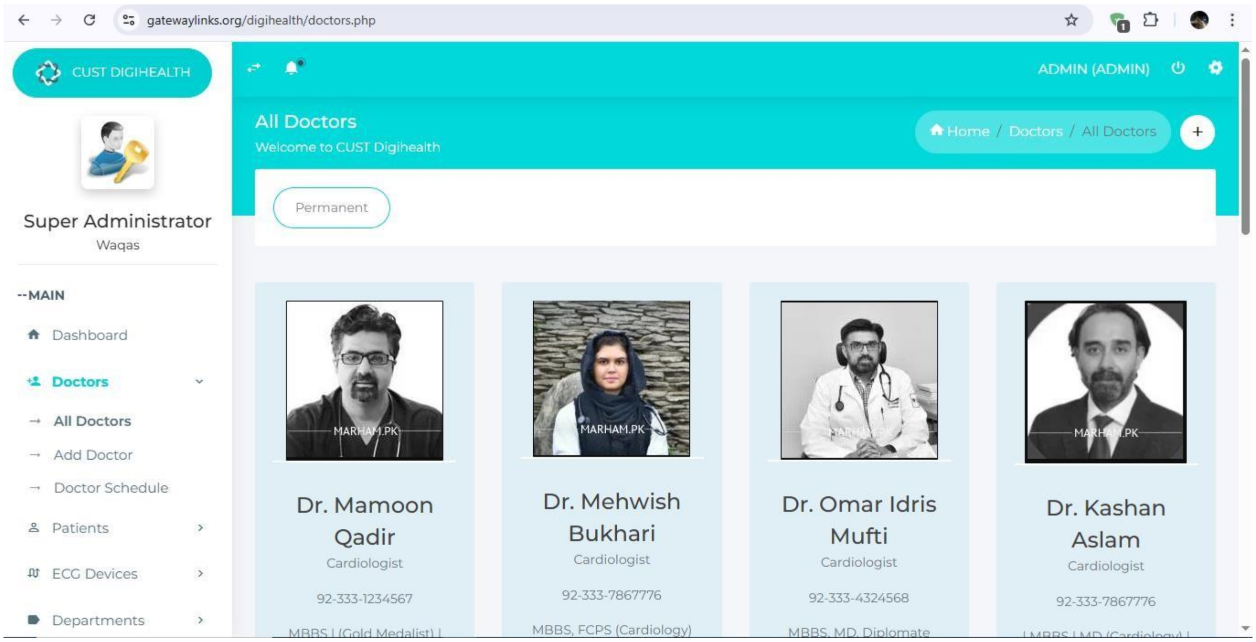
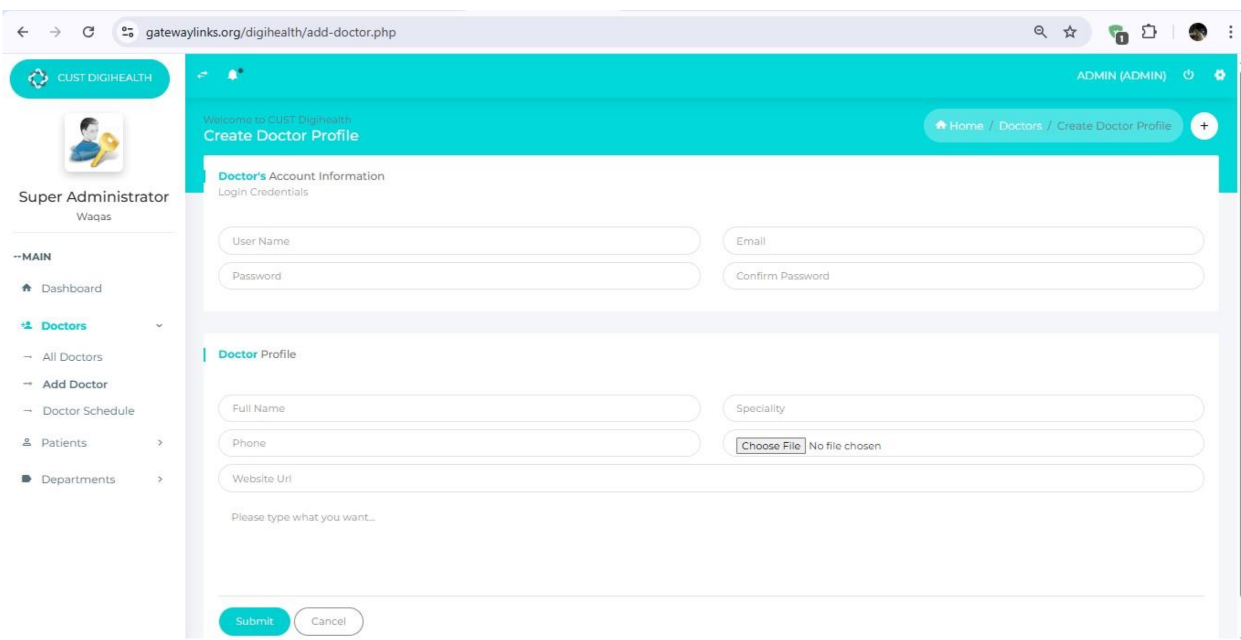


Figure 33 3.7.5 Admin Dashboard overview



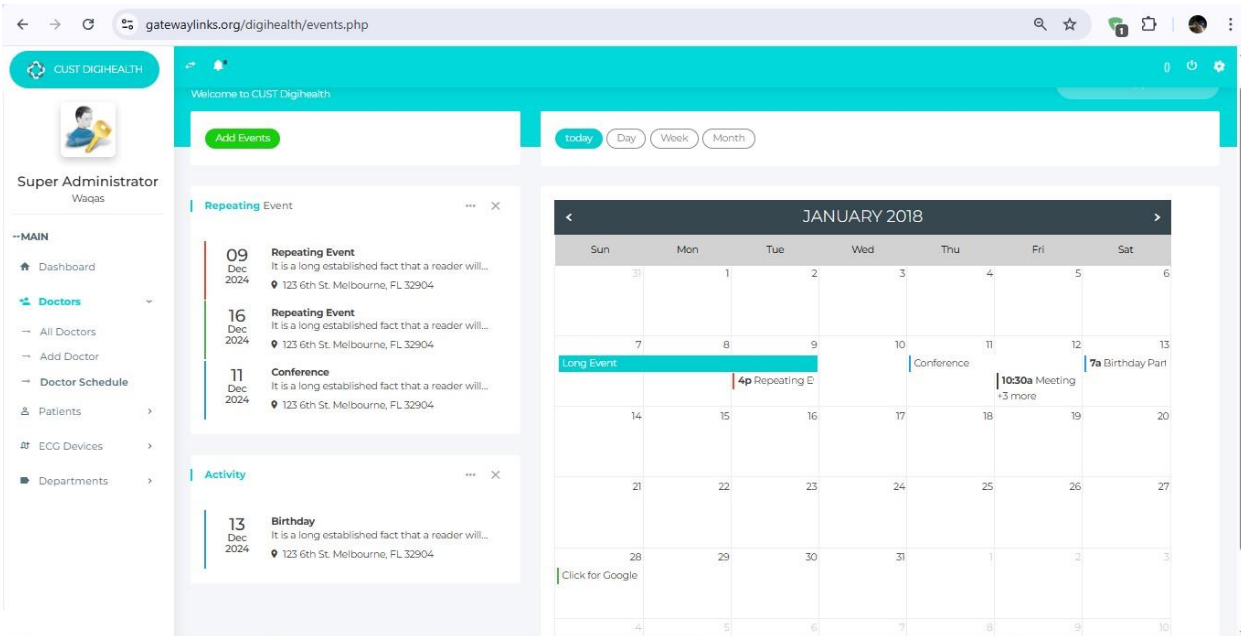
Add doctors :

Figure 34 3.7.6 Add doctors



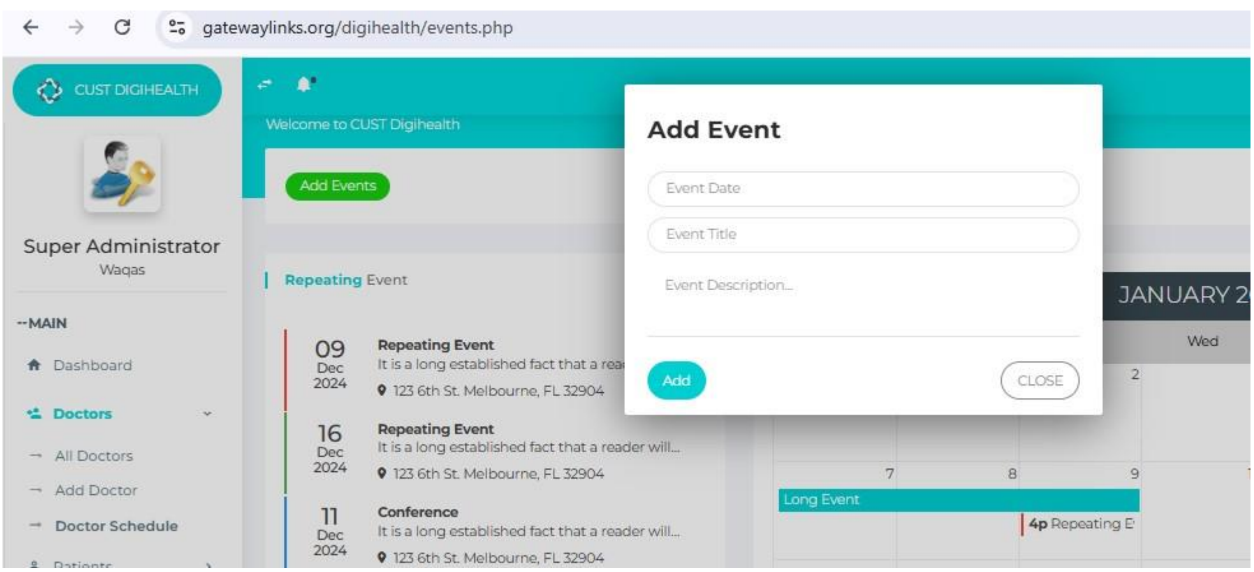
Doctor’s Schedule ;

Figure 35 3.7.7 Doctors Schedule



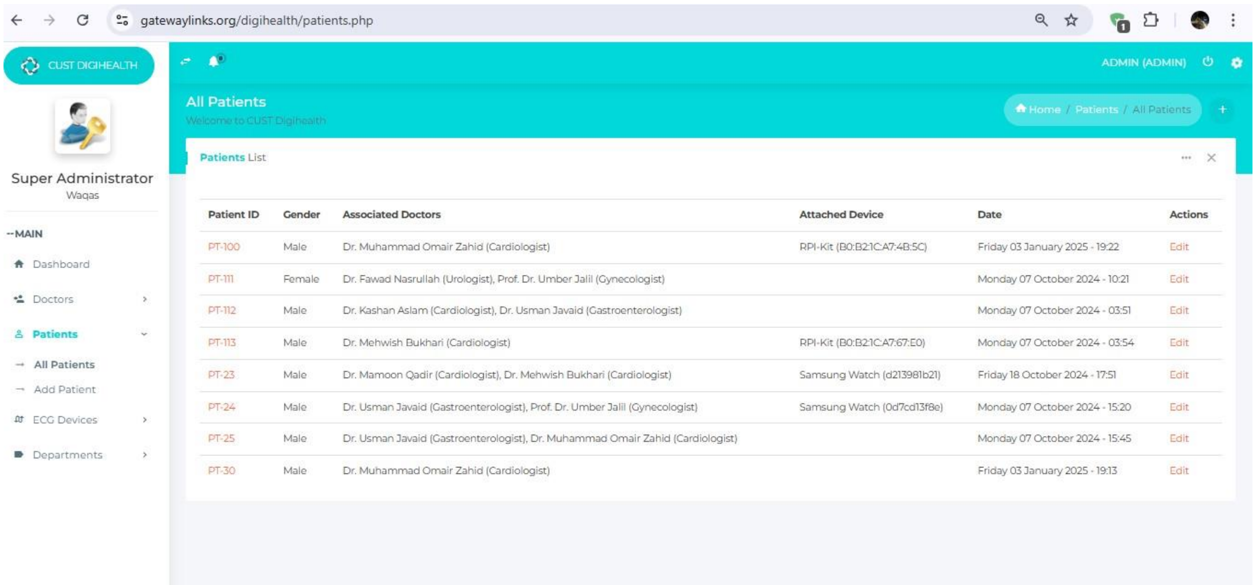
Add Events:

Figure 36 3.7.8 Add events



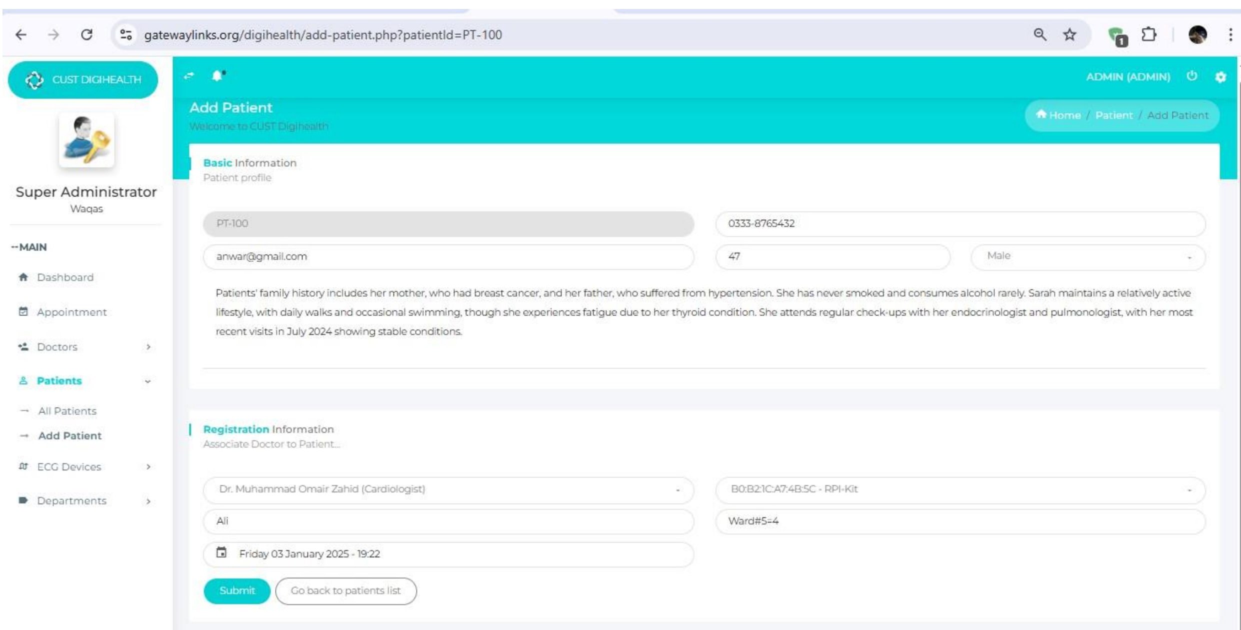
Pateint's :

Figure 37 3.7.9 Patients view



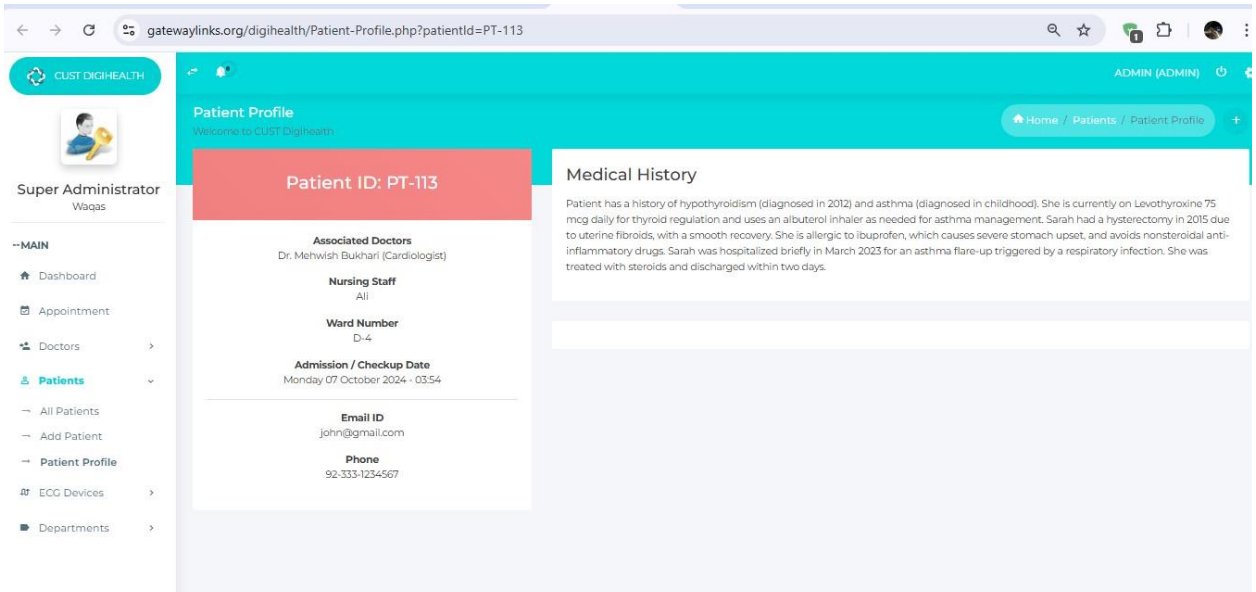
Edit patient :

Figure 38 3.7.10 Edit Patient



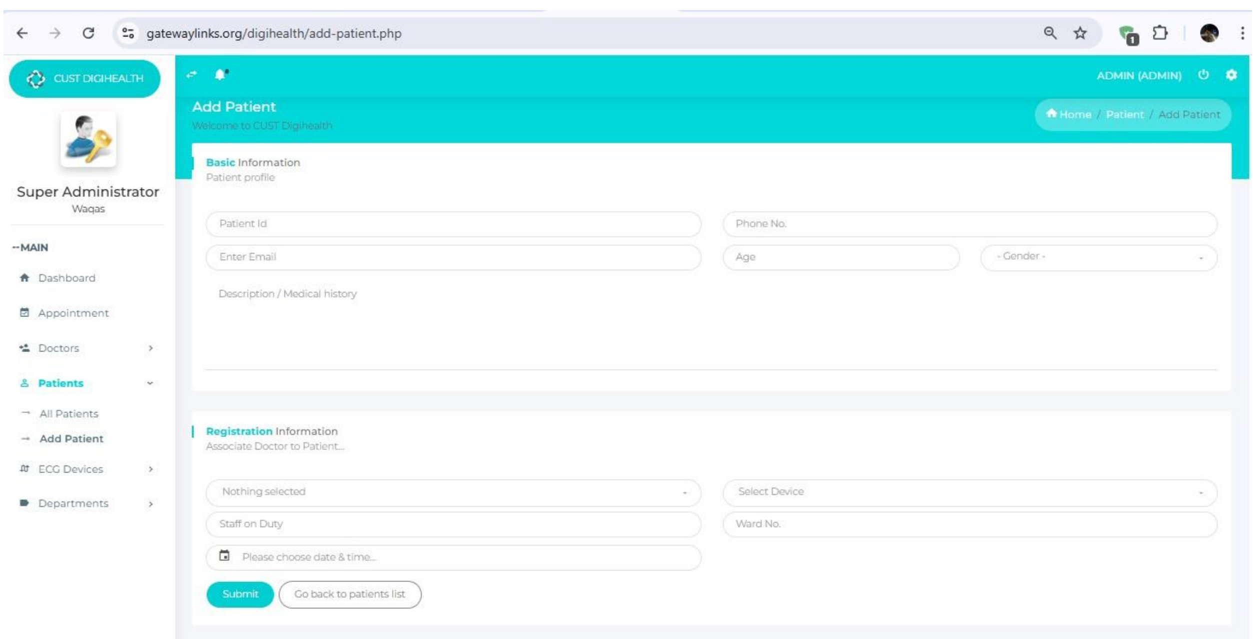
Pateint's Profile :

Figure 39 3.7.11 Patients Profile



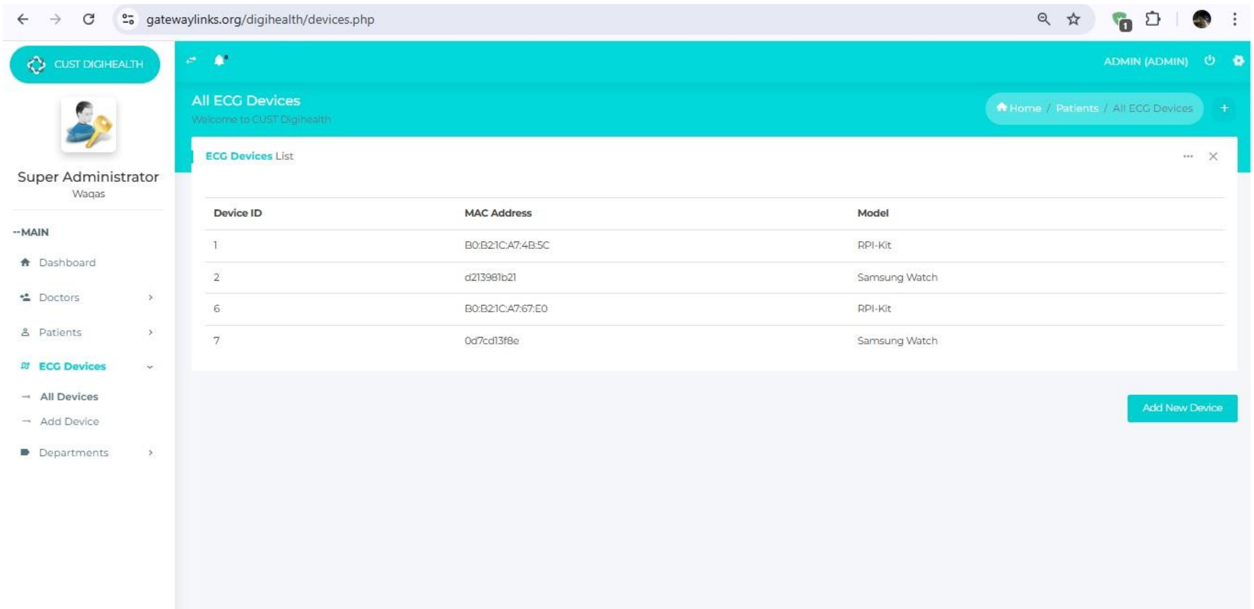
Add Patient's :

Figure 40 3.7.12 add patients



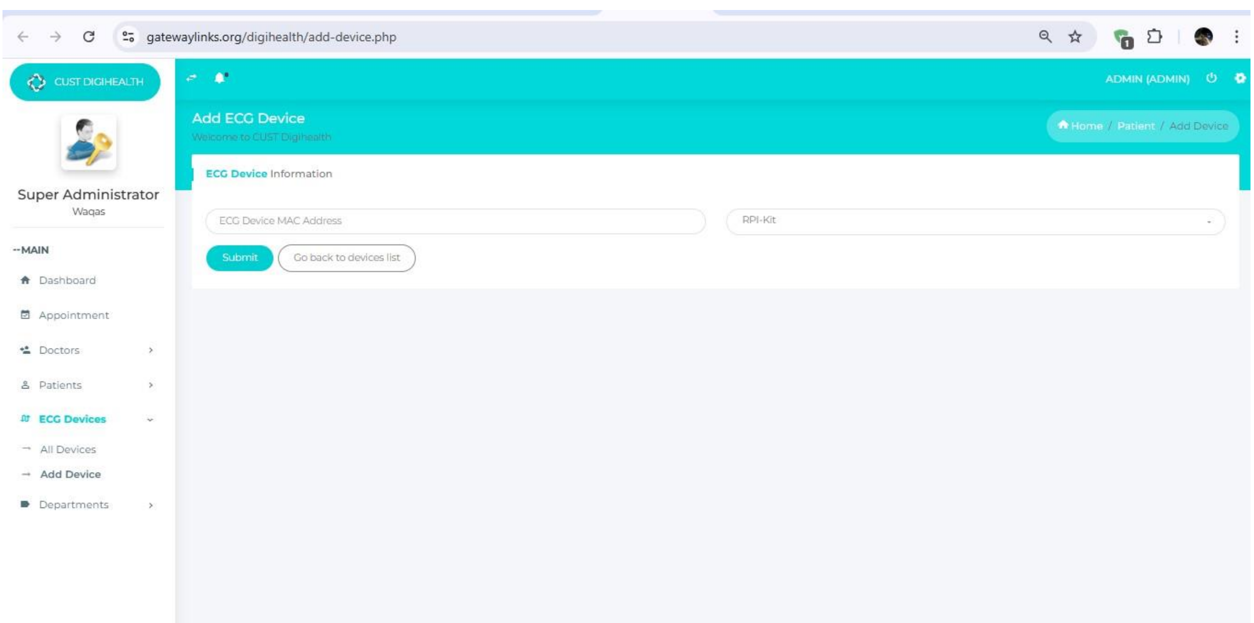
ECG Devices:

Figure 41 3.7.13 ECG Devices



Add Devices:

Figure 42 3.7.14 Add devices



Doctor’s Dashboard:

Figure 43 3.7.15 Doctors Dashboard

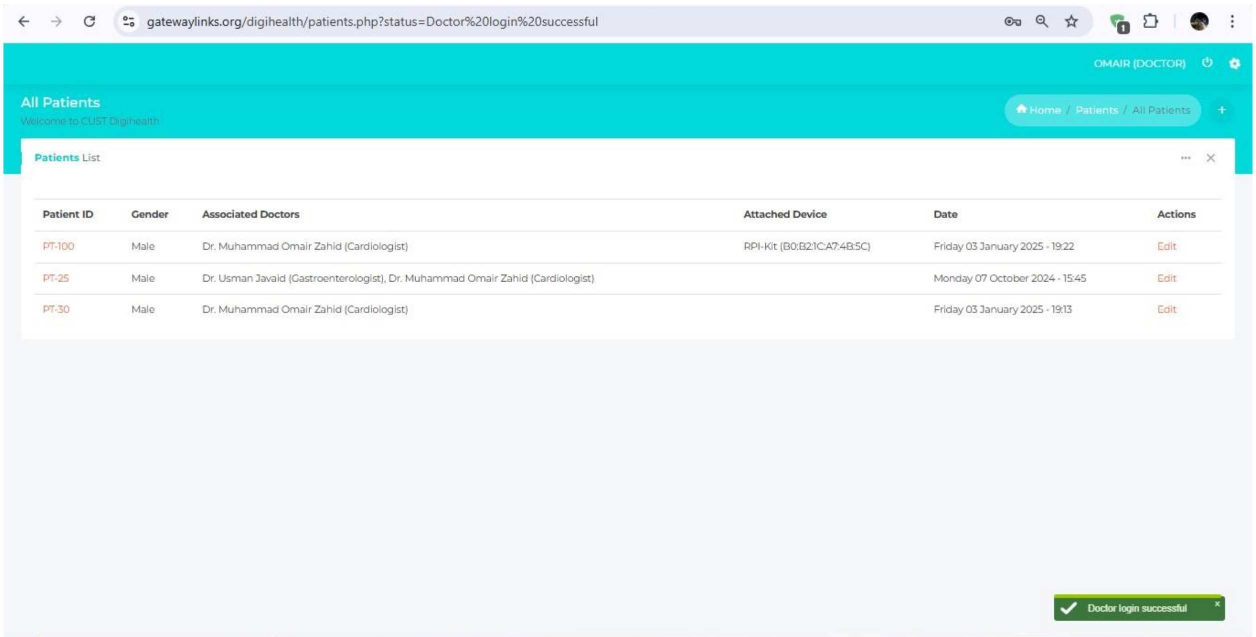


Figure 44 3.7.16 Doctors edit patients

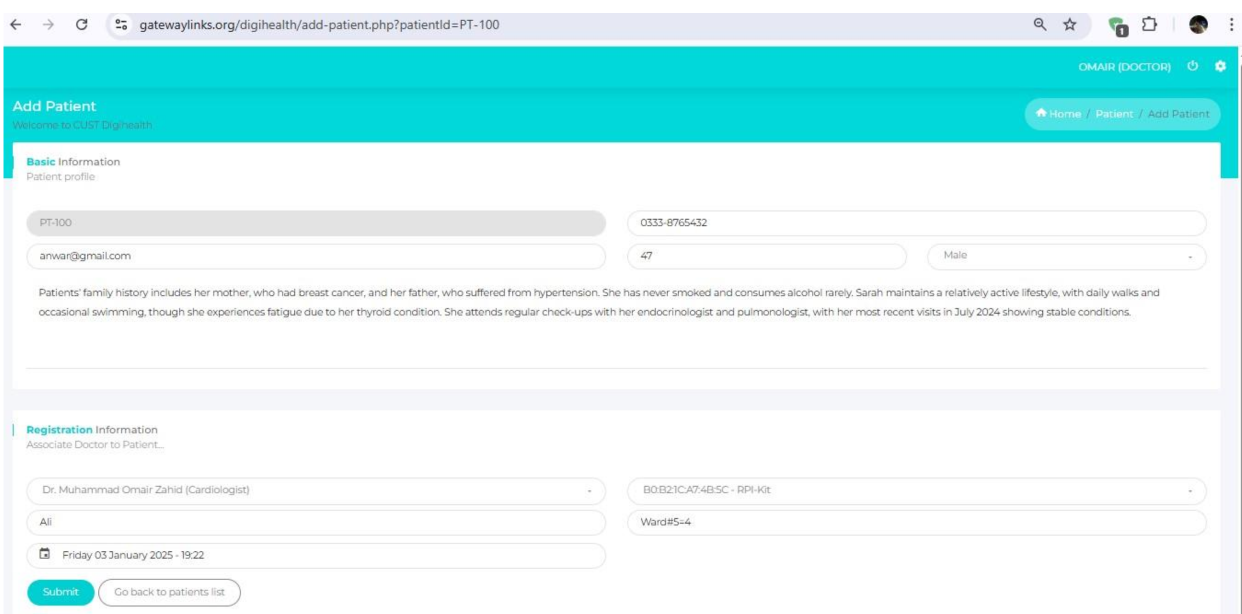
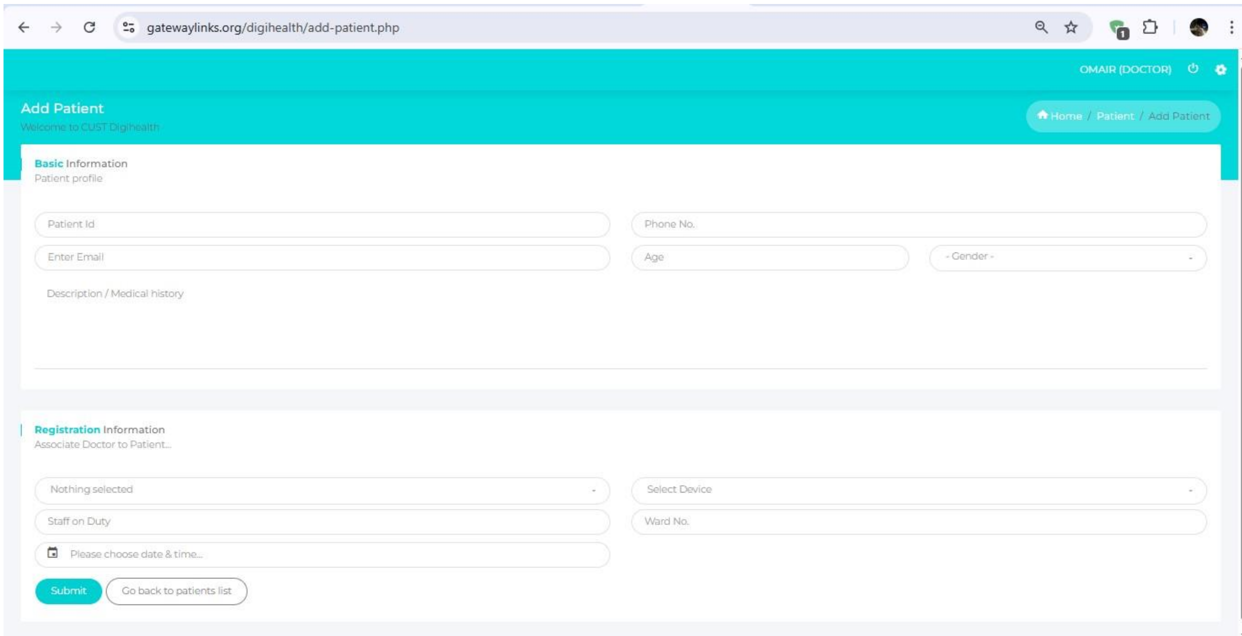
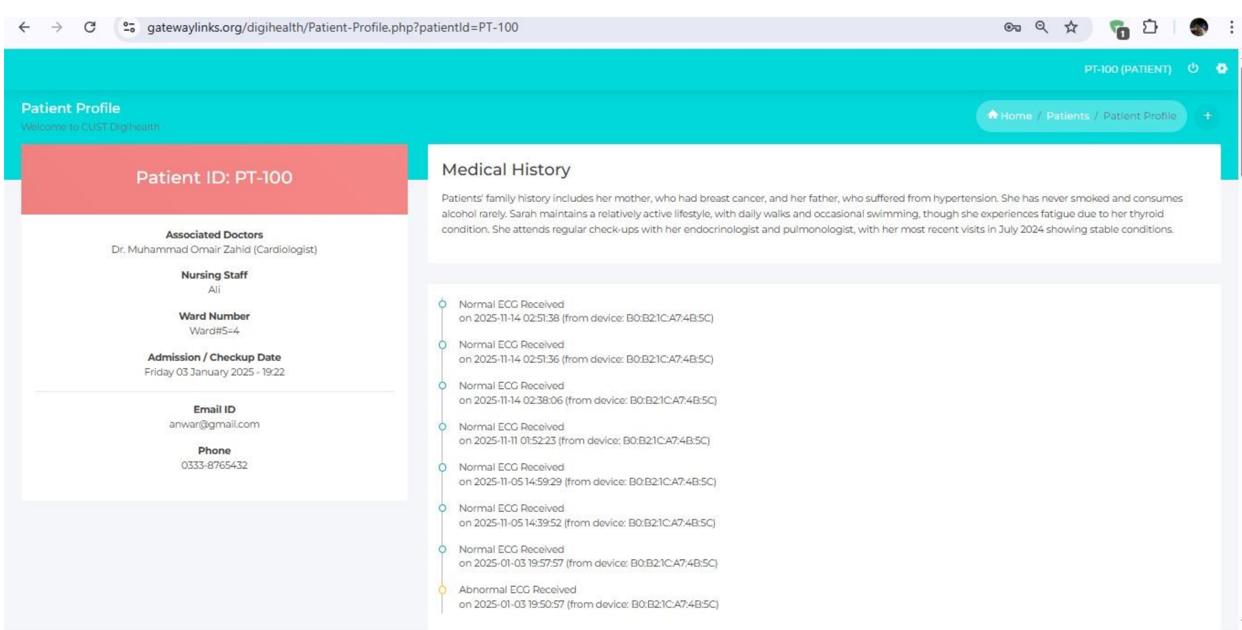


Figure 45 3.7.17 Doctor can add patients



Patient's Dashboard

Figure 46 3.7. 18 Patients Dashboard



Chapter 4: Implementation

The system is implemented using PHP and MySQL. The backend manages authentication, role-based access control, patient records, device registration, and dashboard rendering. The api/ module handles communication with monitoring devices. No AI inference or ADS1292R preprocessing is included in the current phase.

4.1. Development Environment

The development of the AI-Powered Cardiovascular Patient Monitoring System was carried out using PHP as the backend programming language and MySQL as the database management system. XAMPP was used as the local development environment, providing Apache server and MySQL database services. The frontend interfaces were developed using HTML, CSS, and JavaScript. Raspberry Pi devices are supported as monitoring gateways for transmitting physiological data to the web application.

4.2. System Folder Structure

The system follows a modular folder structure. The api directory handles communication between monitoring devices and the backend system. The assets directory contains CSS, JavaScript, and library files used for the user interface. Backend logic for authentication, patient management, and data handling is implemented using PHP scripts.

4.3. Backend Implementation (PHP)

The backend of the system is implemented using PHP. It handles user authentication, role-based access control, patient profile management, device registration, and retrieval of monitoring data. Sessions are used to maintain secure user access across the system.

4.4. Database Implementation (MySQL)

MySQL is used as the database management system. The database stores user credentials, patient profiles, doctor records, device information, and physiological readings. The database schema is designed to support future expansion for AI prediction logs.

4.5. User Interface Implementation

4.5.1 Admin Dashboard Section

Admin Login Page

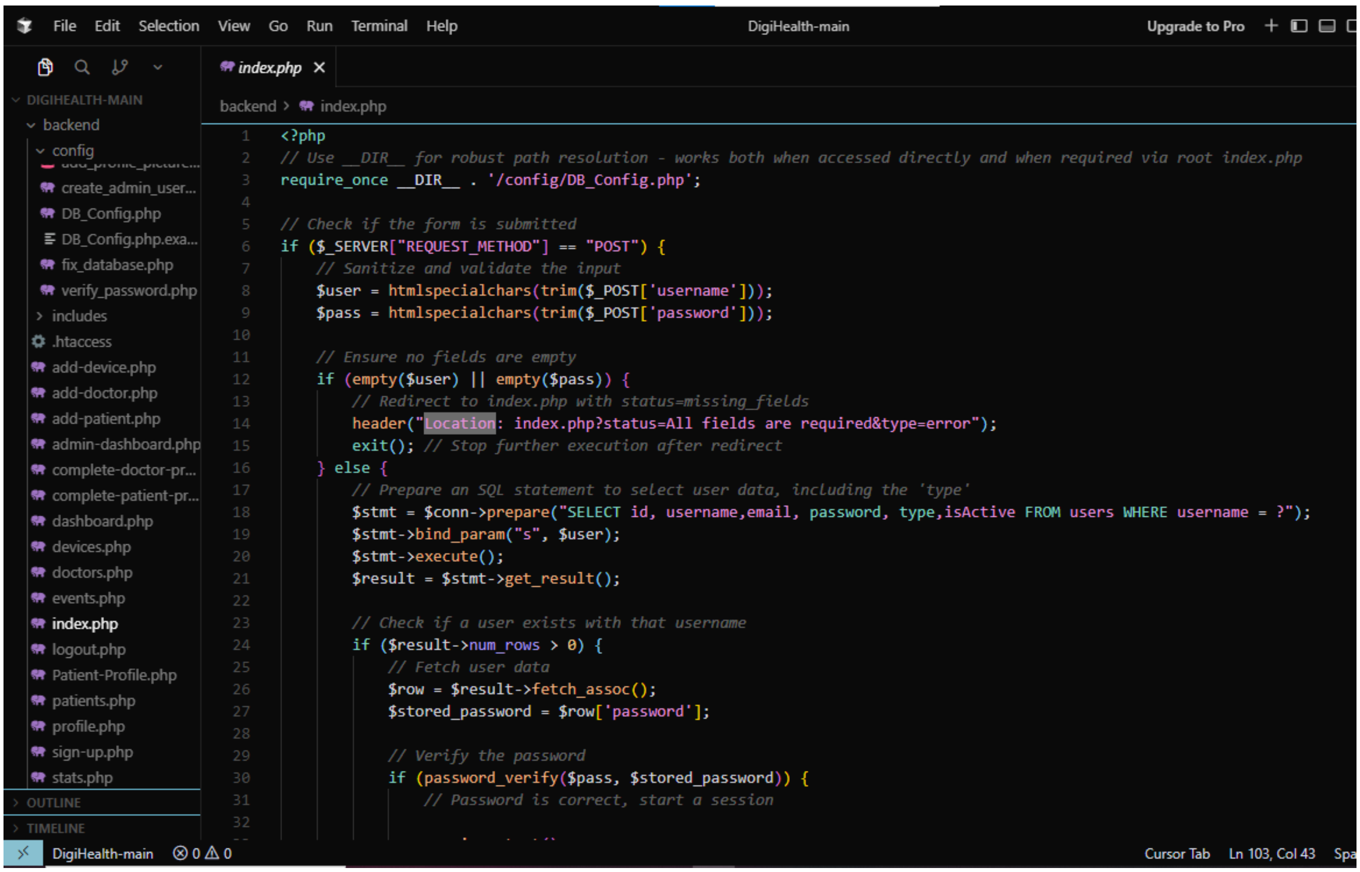


Figure 47 4.5.1 Admin login page

Admin Dashboard

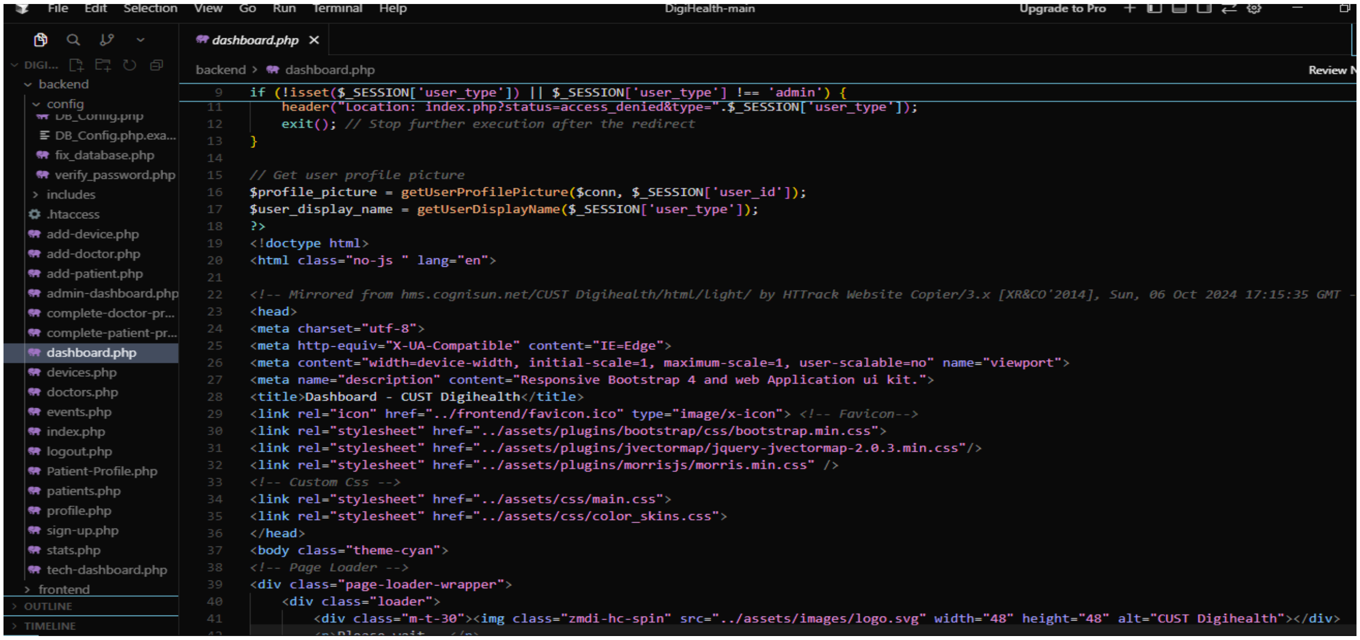


Figure 48 4.5.2 admin dashboard

Add Doctor Page

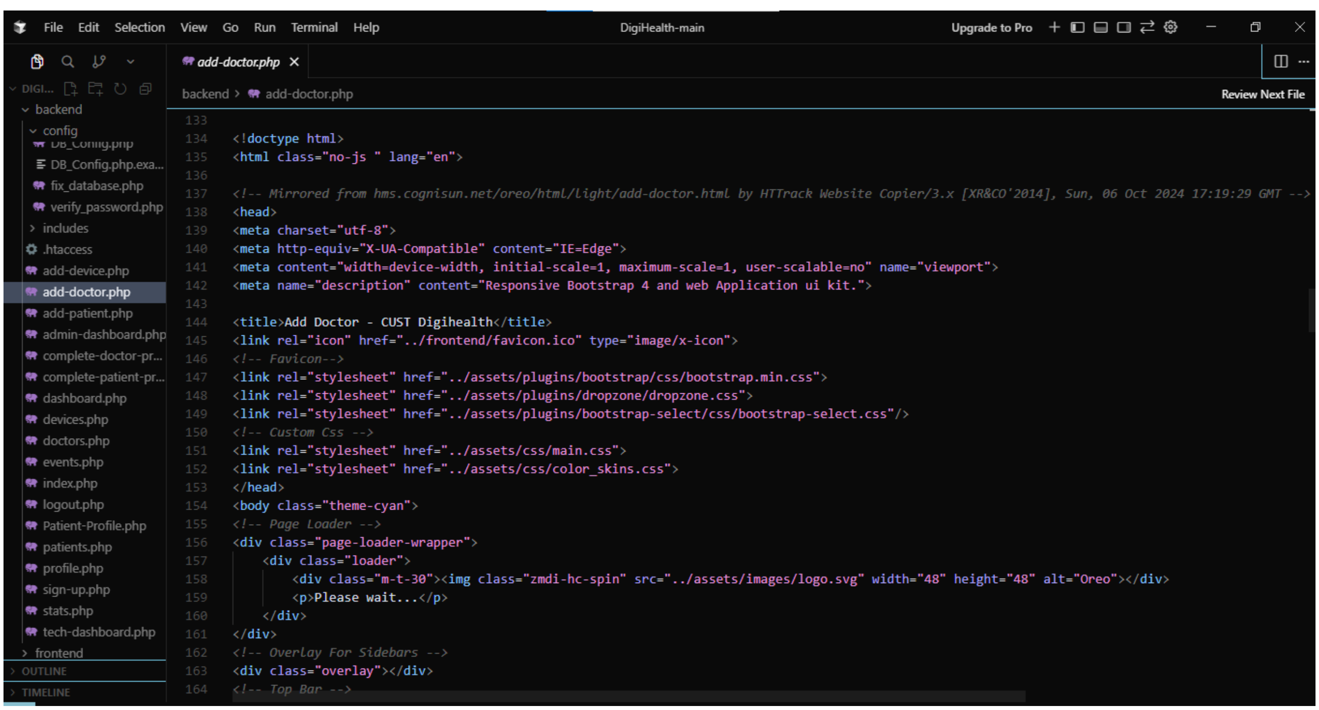


Figure 49 4.5.3 Add doctor page

View All Doctors

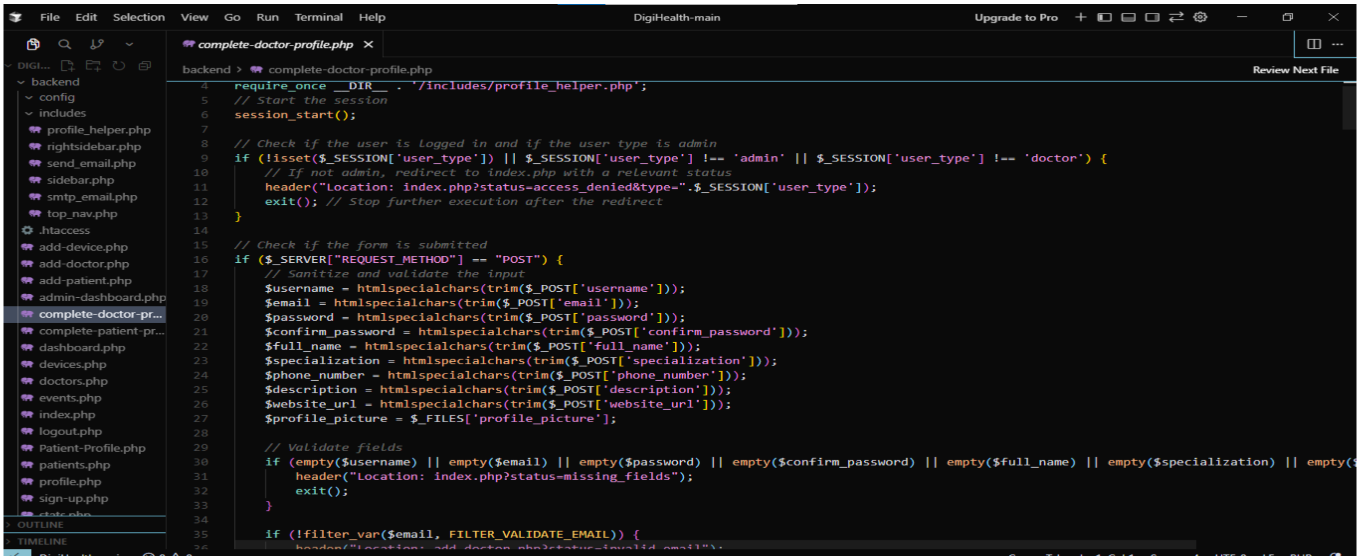


Figure 50 4.5.4 view doctors

Add Device Page

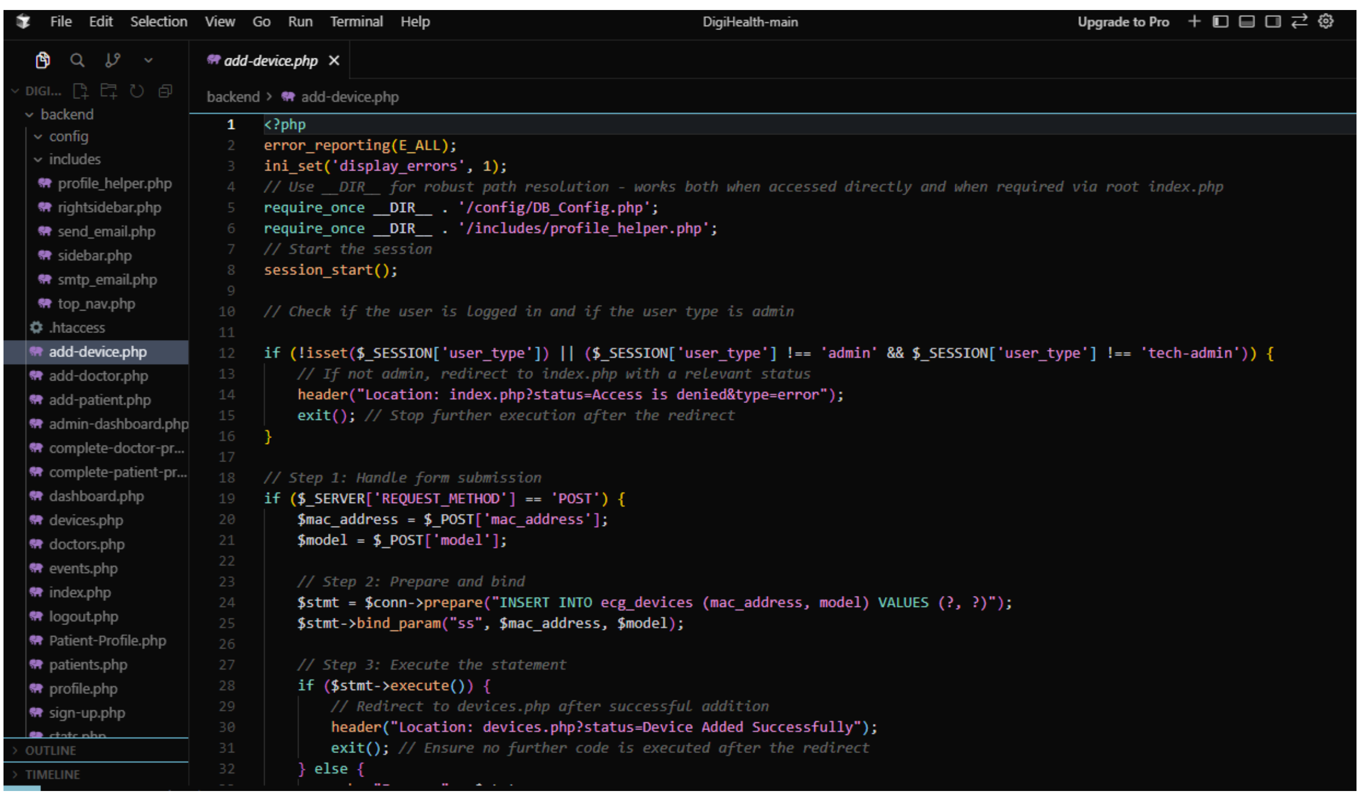


Figure 51 4.5.5 add device page

Medical History Page

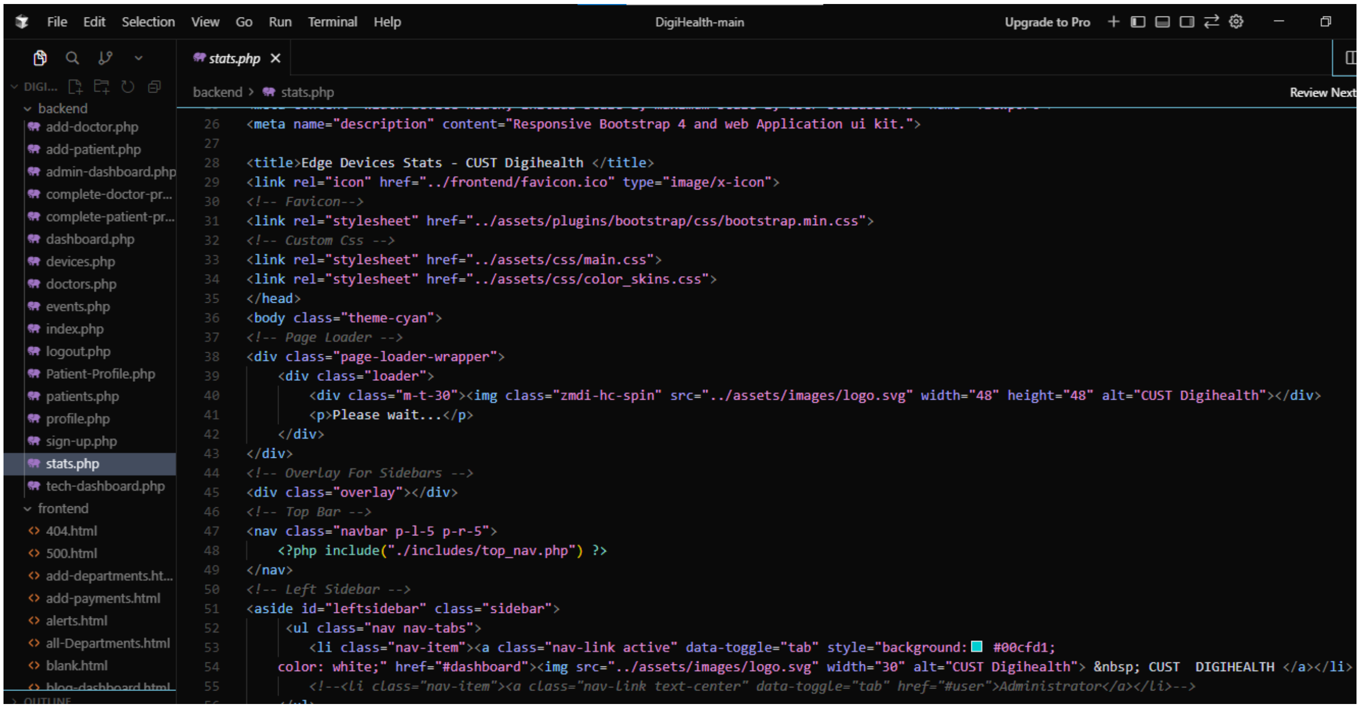


Figure 52 4.5.6 medical history

API Module Structure

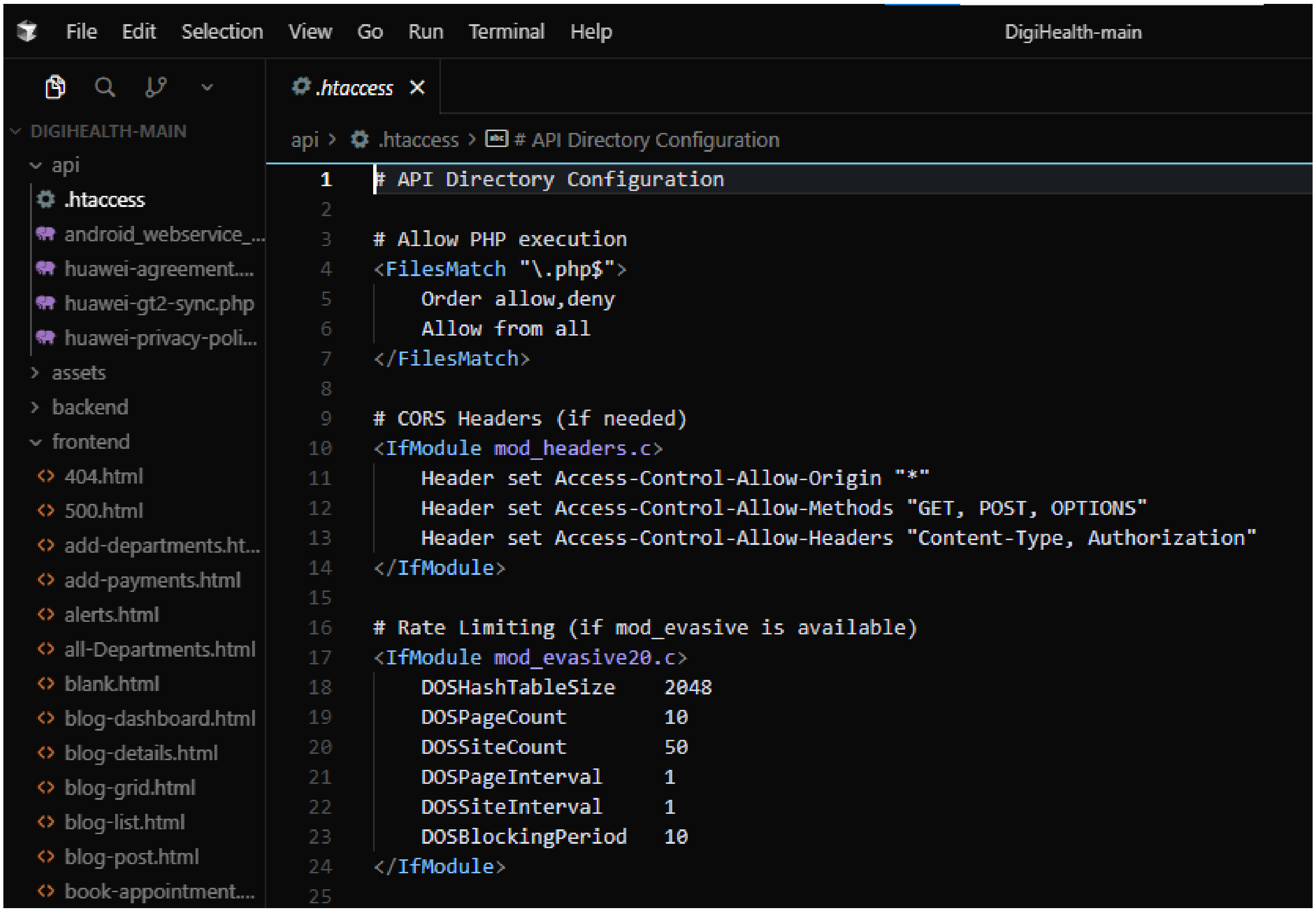


Figure 53 4.5.7 API Modules

The API module handles communication between monitoring devices and the web application.

4.6. Device Integration

The system supports Raspberry Pi devices as monitoring gateways. Devices can be registered and managed through the admin interface. Sensor-level signal preprocessing and medical-grade ECG acquisition using ADS1292R are planned for future implementation.

Chapter 5: Software Testing

5.1. Testing Methodology

The system was tested using **Black-Box Testing methodology**. In black-box testing, the internal code structure is not considered. Instead, the system is evaluated based on input-output behavior and requirement compliance.

Test cases were designed according to the functional requirements defined in Chapter 2. Each test case includes:

- Test Case ID
- Feature Tested
- Preconditions
- Test Steps
- Expected Result
- Actual Result
- Status

This approach ensures that the system behaves correctly from a user's perspective.

5.2. Test Environment

Testing was performed on a local development environment using XAMPP and MySQL. The system was accessed using standard web browsers. Raspberry Pi devices were tested for connectivity and data transmission through APIs.

Testing was performed under the following environment:

- **Web Server:** PHP Development Server (localhost:8000)
- **Database:** MySQL (gatewayl_ecg_db)
- **Development Stack:** XAMPP (Apache + MySQL)
- **Backend Language:** PHP

- **Frontend Technologies:** HTML, CSS, JavaScript
- **Browser:** Google Chrome
- **Tool:** Vs Code / Cursor AI

The database schema was imported successfully, and the configuration file (DB_Config.php) was verified for correct credentials before testing.

5.3. Test Cases

Functional testing verifies that each implemented module performs according to system requirements.

5.3.1 Test Case 1 – User Login

Table 19 5.3.1 TC

Field	Description
Test Case ID	TC-01
Feature Tested	User Authentication
Preconditions	User account exists and is active
Test Steps	1. Open login page 2. Enter valid username and password 3. Click Login
Expected Result	User is redirected to the appropriate dashboard
Actual Result	Successful redirection based on role
Status	Pass

Additional validation:

- Invalid password → Error message displayed
- Inactive user → Access denied

5.3.2 Test Case 2 – Role-Based Access Control (RBAC)

Table 20 5.3.2 TC 2

Field	Description
Test Case ID	TC-02
Feature Tested	Role-Based Authorization
Preconditions	User logged in
Test Steps	1. Login as Admin 2. Attempt to access Doctor page 3. Login as Doctor 4. Attempt to access Admin panel
Expected Result	Unauthorized pages are restricted
Actual Result	Access correctly restricted
status	Pass

The system successfully prevents unauthorized access and redirects users to login if sessions are not valid.

5.3.3 Test Case 3 – Add Patient

Table 21 5.3.3 TC 3

Field	Description
Test Case ID	TC-03
Feature Tested	Patient Registration
Preconditions	Doctor logged in
Test Steps	1. Navigate to Add Patient page 2. Enter patient details 3. Submit form
Expected Result	Patient record stored in database
Actual Result	Patient successfully added
Status	Pass

Form validation was also tested for missing required fields.

5.3.4 Test Case 4 – Device Registration

Table 22 5.3. 4 TC 4

Field	Description
Test Case ID	TC-04
Feature Tested	Device Registration
Preconditions	Admin logged in
Test Steps	1. Navigate to Add Device page 2. Enter device details 3. Submit
Expected Result	Device stored in database
Actual Result	Device successfully registered
Status	Pass

Duplicate device IDs were tested and correctly rejected.

5.3.5 Test Case 5 – Dashboard Loading

Table 23 5.3.5 TC 5

Field	Description
Test Case ID	TC-05
Feature Tested	Dashboard Visualization
Preconditions	User logged in
Test Steps	1. Login to system 2. Redirect to dashboard
Expected Result	Dashboard loads and displays relevant data
Actual Result	Dashboard displayed correctly
Status	Pass

5.4. Testing Summary

The results of the testing phase indicate that the implemented modules of the system function correctly and meet the defined functional requirements for the current development phase.

The authentication system, role-based access control, patient management module, device registration module, and dashboard visualization are stable and operational.

The system is ready for demonstration and further enhancement in subsequent development phases.